

Estructuras de Datos

Clase 20a – Árboles binarios de búsqueda



Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



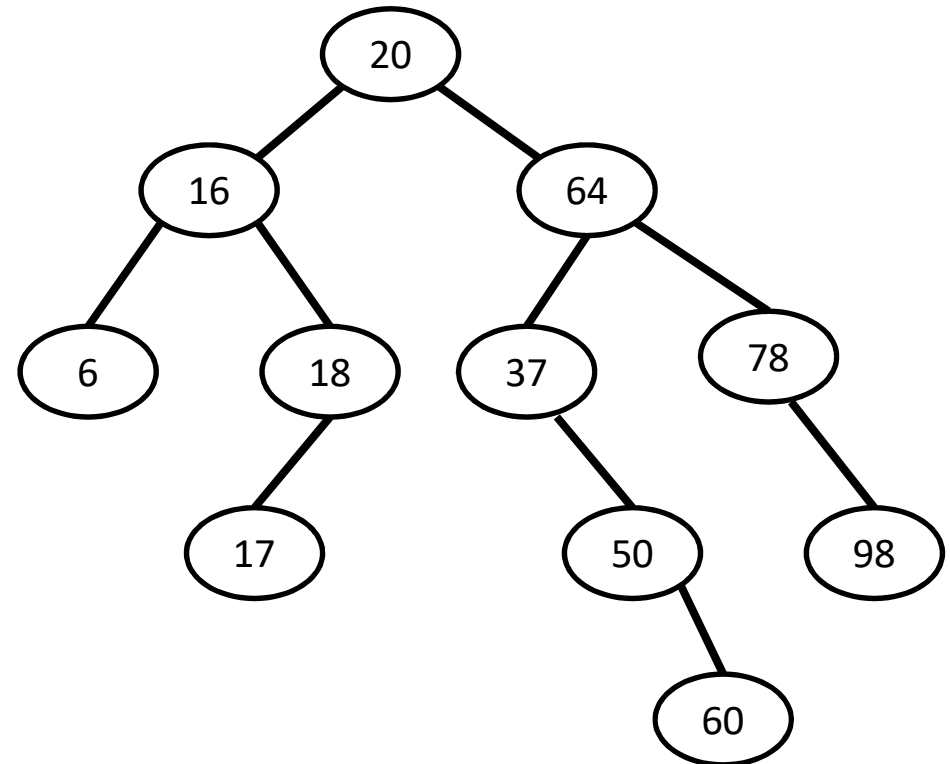
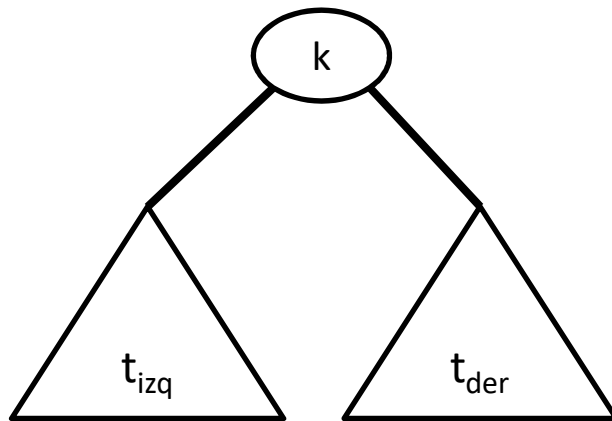
Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Motivaciones

- El árbol binario de búsqueda (ABB) es una estructura de datos útil para implementar conjuntos, mapeos y diccionarios.
- En un ABB las claves en los nodos se hallan ordenadas de una manera particular.
- El tiempo de insertar, recuperar y eliminar es proporcional a la altura del ABB.
- Si el árbol tiene n claves, la altura del ABB se halla entre $\log_2(n)$ y n .

Definición

- Un ABB es un árbol binario tal que:
 - Es vacío, o,
 - Es un nodo con rótulo k e hijos t_{izq} y t_{der} tales:
 - $k >$ claves de t_{izq} ,
 - $k <$ claves de t_{der} , y,
 - t_{izq} y t_{der} son ABB.



Búsqueda de una clave k partir del nodo p

Algoritmo `Buscar(  $k$ ,  $p$  )` // La primera vez se llama con la raíz del ABB

Si p es vacío entonces // CB: Se me terminó el árbol

return falso // k no se encuentra en el ABB

Sino si $k = \text{clave}(p)$ entonces // CB: encontré a k

return true // k sí se encuentra en el ABB porque lo encontré

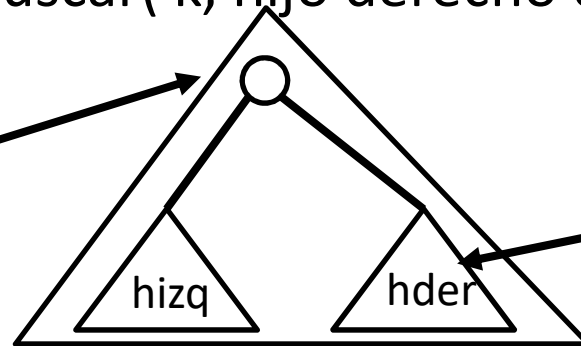
Sino si $k < \text{clave}(p)$ entonces // CR: k es menor a la clave de p

return `Buscar(  $k$ , hijo izquierdo de  $p$  )`

Sino // $k > \text{clave}(p)$: CR: k es mayor a la clave de p

return `Buscar(  $k$ , hijo derecho de  $p$  )`

Subárbol
del ABB
con raíz p



Si el $k > \text{clave}(p)$, tengo que buscar a k en el hijo de derecho de p , que también (conceptualmente) es un ABB porque descarté a la raíz y al otro subárbol (hijo izquierdo de p).

Inserción en un ABB

- Las nuevas claves siempre se insertan como hijo de un nodo hoja.
- Algoritmo insertar(k, p) { Comienza en la raíz del ABB }

Si p es vacío entonces

 crear un nodo hoja con rótulo k

Sino

 si $k < \text{clave}(p)$ entonces

 insertar(k, hijo izquierdo de p)

 sino si $k > \text{clave}(p)$ entonces

 insertar(k, hijo derecho de p)

 sino si $k = \text{clave}(p)$ entonces

 reemplazar rótulo de p

Nota: La operación “reemplazar rótulo de p” dependerá de si:

- (a) Implemento un conjunto: no hacer nada
- (b) Implemento un mapeo: modifico el valor de la entrada
- (c) Implemento un diccionario: Agrego una entrada con clave k

Ejemplos de inserciones

- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

- Ejemplo 2: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

2, 4, 6, 9, 10, 34, 78, 90, 100, 120

- Ejemplo 3: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

120, 100, 45, 34, 29, 26, 16, 13, 5, 2

- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

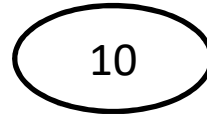
10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓

Comenzamos en el
árbol vacío
insertando 10



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓

10

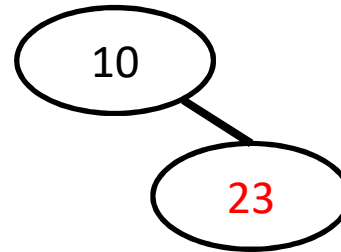
Queremos insertar
23, como $23 > 10$,
vamos hacia la
derecha.

- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓

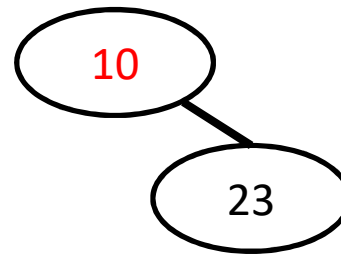
Como llegamos a un
nodo vacío allí va el
23 y terminamos.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
 ✓ ✓

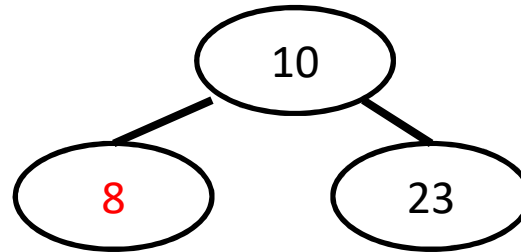
Ahora queremos
insertar el 8.
Como $8 < 10$, vamos
hacia la izquierda.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
 ✓ ✓ ✓

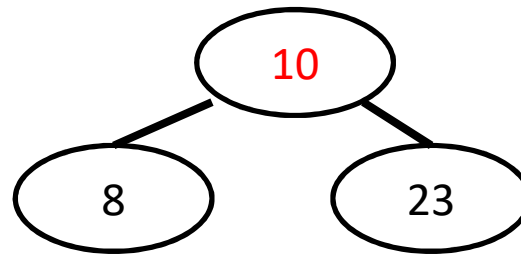
Como llegamos a un
nodo vacío, allí va el
8 y terminamos.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
 ✓ ✓ ✓

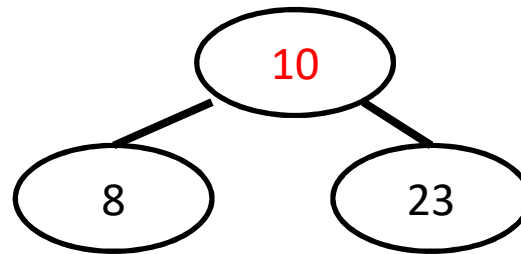
90 > 10 => vamos a
la derecha.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
 ✓ ✓ ✓

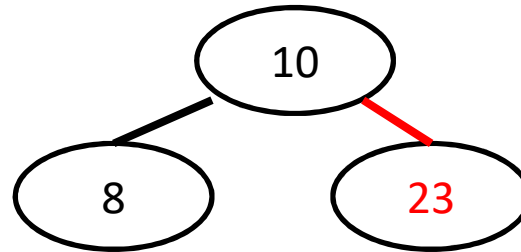
90 > 10 => vamos a
la derecha.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
 ✓ ✓ ✓

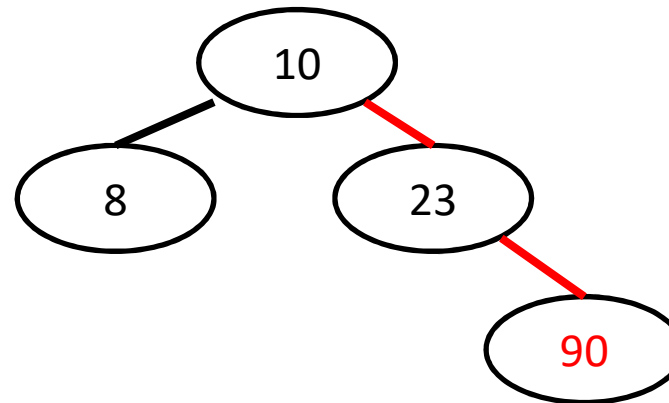
90 > 23 => vamos a
la derecha.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
√ √ √ √

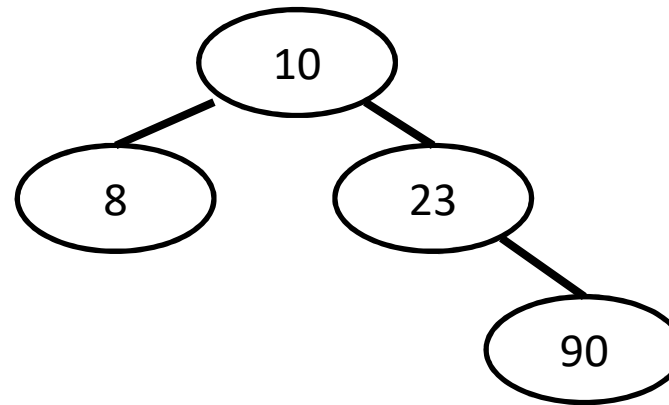
Como llegamos a un
nodo vacío,
terminamos e
insertamos allí el 90
en un nodo nuevo.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓

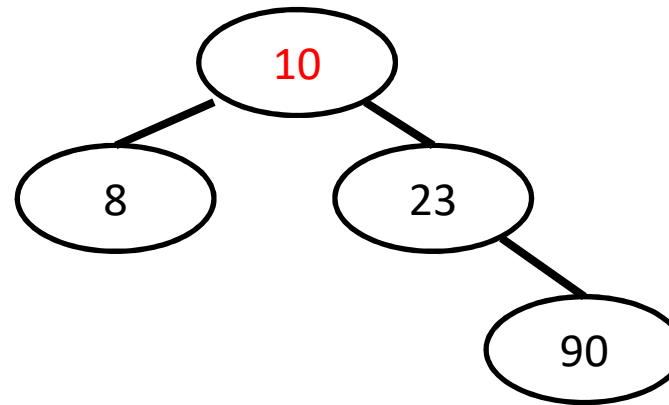
Ahora queremos
insertar el 78.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
√ √ √ √

Como $78 > 10$,
vamos a la derecha.

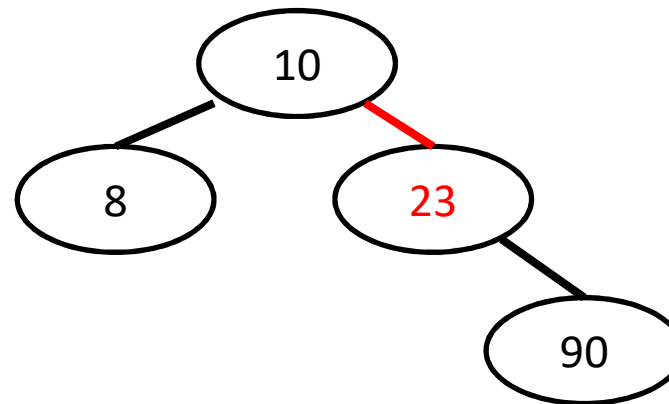


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓

Como $78 > 23$,
vamos a la derecha.

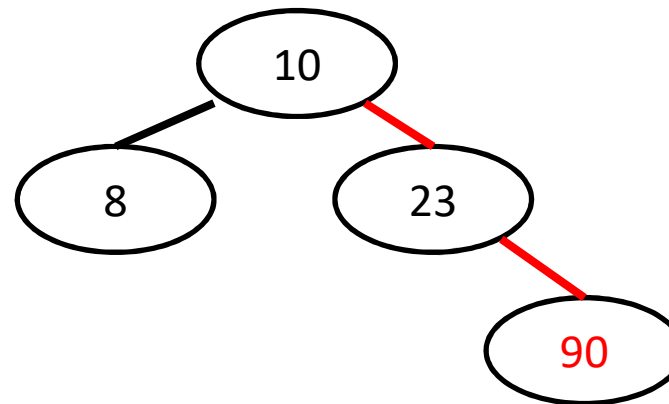


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓

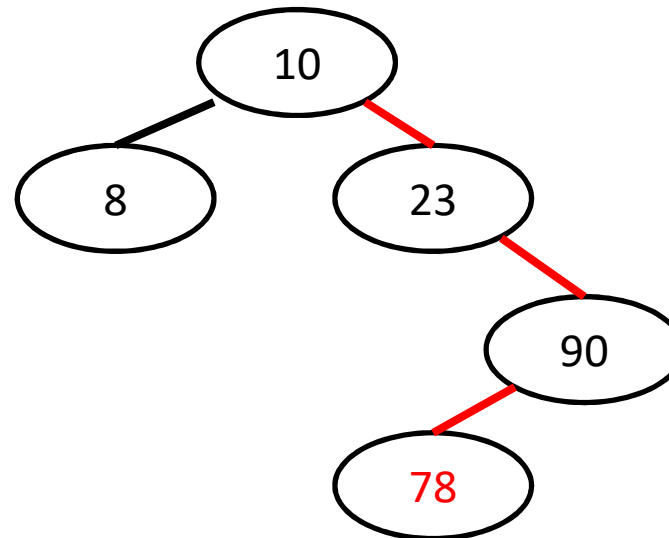
Como $78 < 90$,
vamos a la izquierda.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓

Como llegamos a un
nodo vacío,
insertamos ahí el 78
y terminamos.

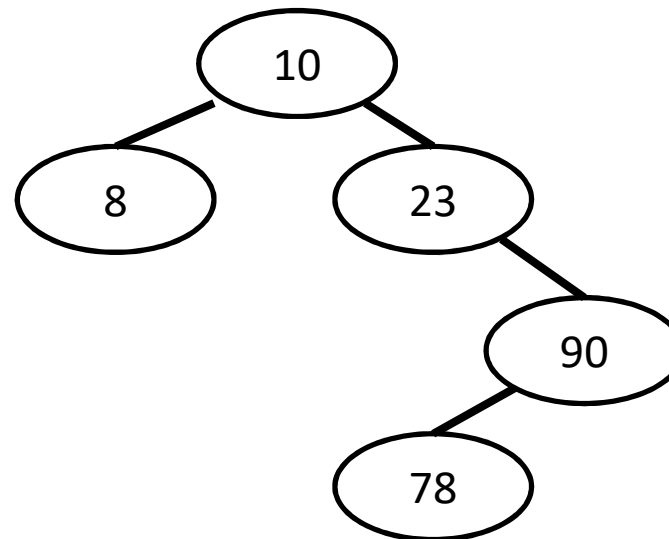


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓

Ahora queremos
insertar el 45.

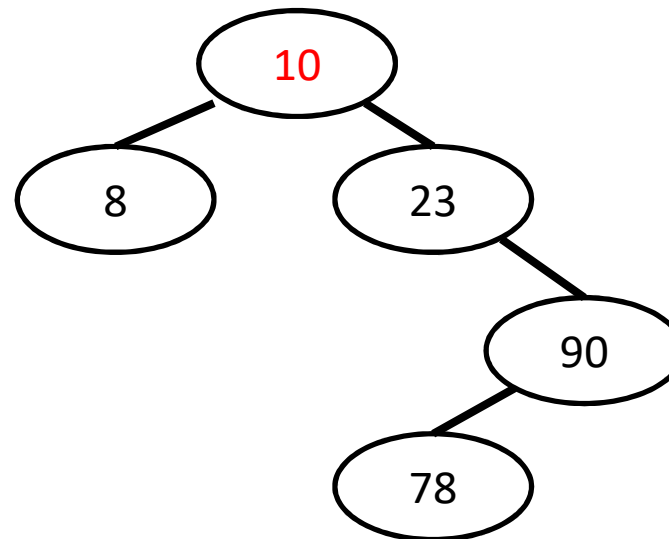


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓

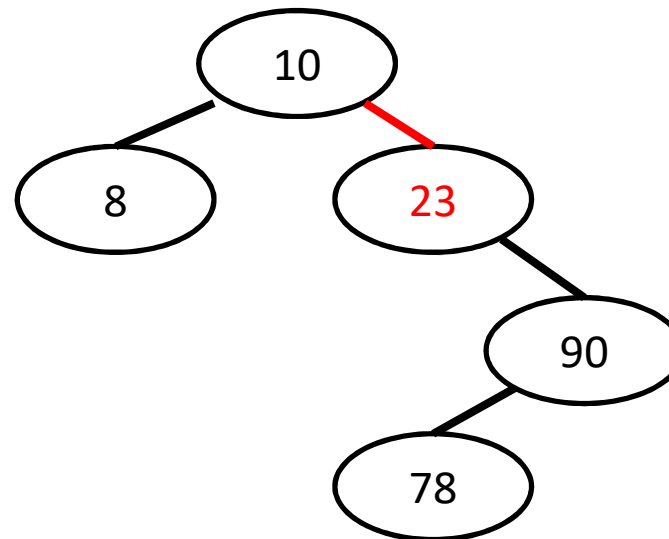
Como $45 > 10$,
vamos a la derecha.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓

Como $45 > 23$,
vamos a la derecha
nuevamente.

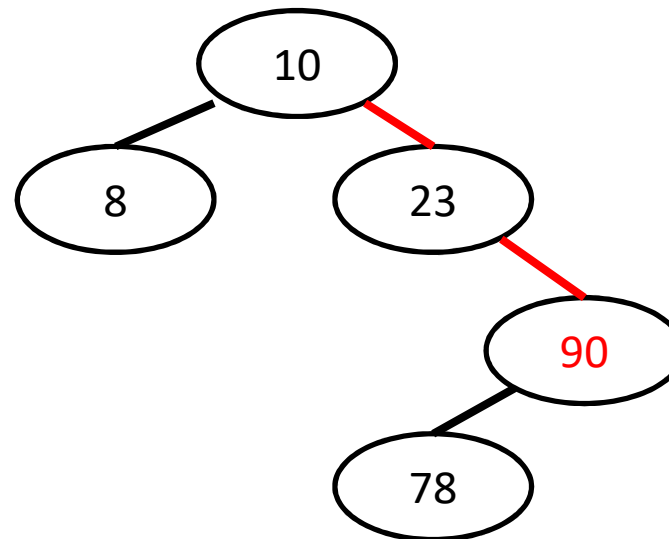


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓

Como $45 < 90$,
vamos a la izquierda.

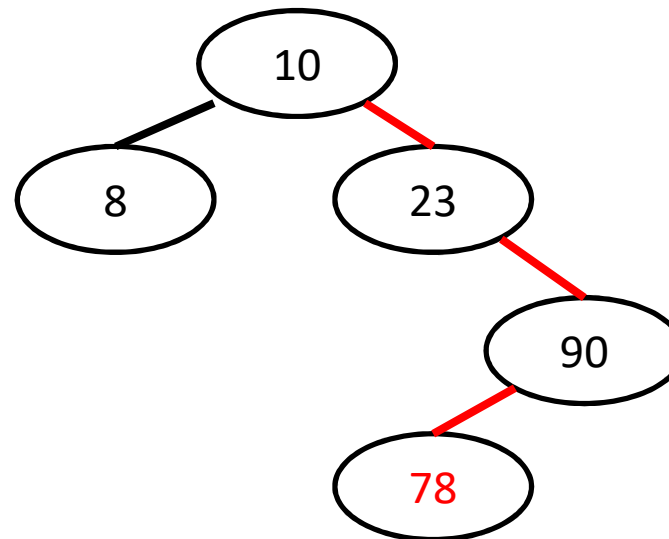


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓

Como $45 < 78$,
vamos a la izquierda
de nuevo.

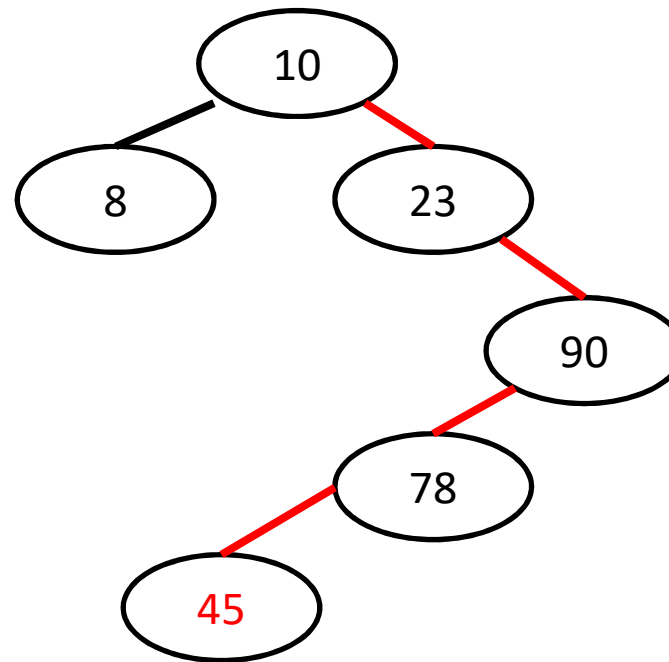


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓

Llegamos a un nodo vacío, insertamos ahí al 45 y terminamos.

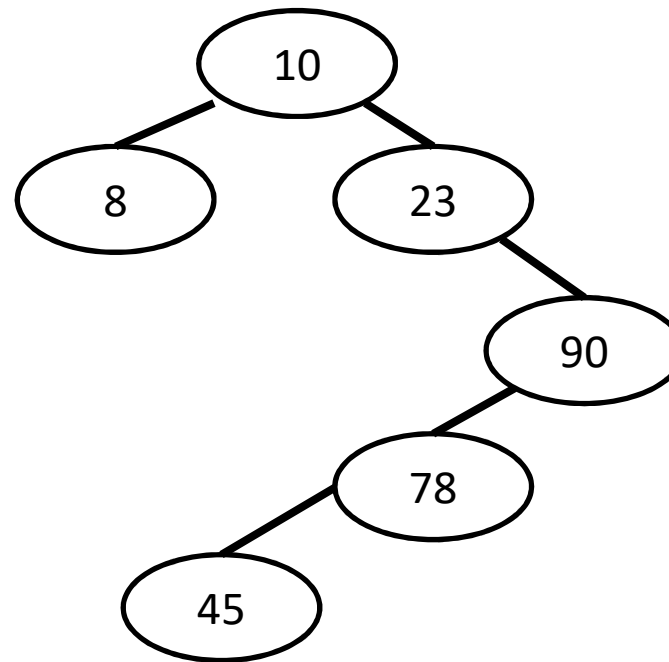


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓

Ahora queremos
insertar el 3.

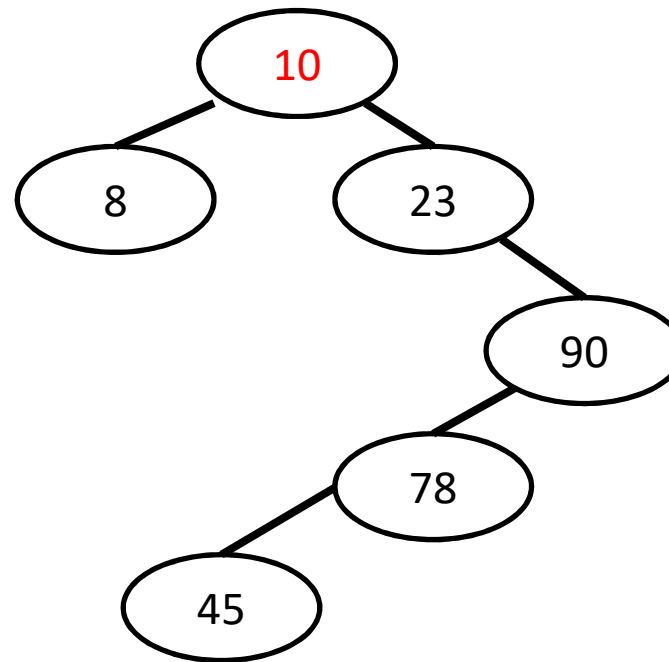


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓

Como $3 < 10$, vamos
hacia la izquierda.

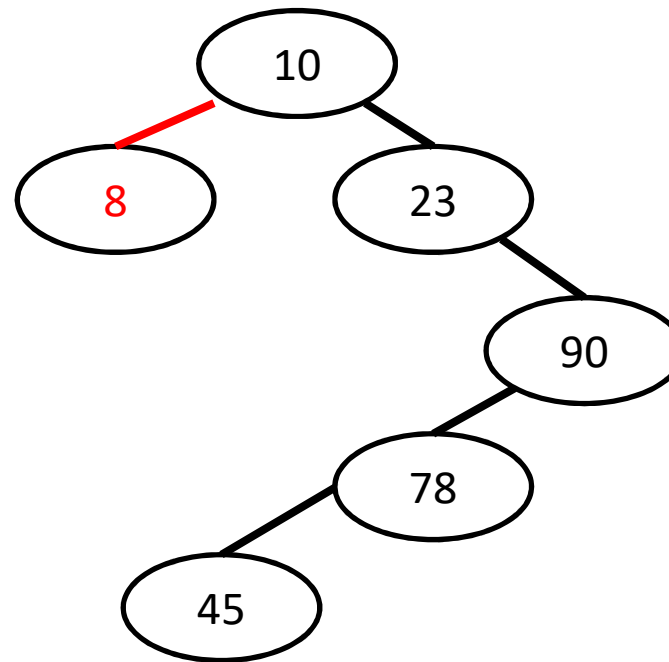


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓

Como $3 < 8$, vamos
hacia la izquierda de
nuevo.

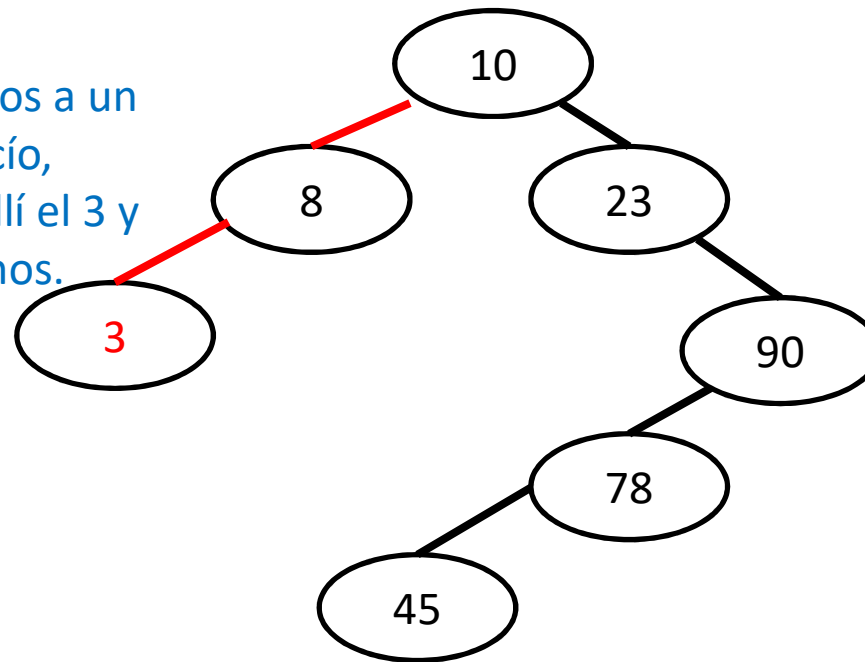


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓

Como llegamos a un
nodo vacío,
insertamos allí el 3 y
terminamos.

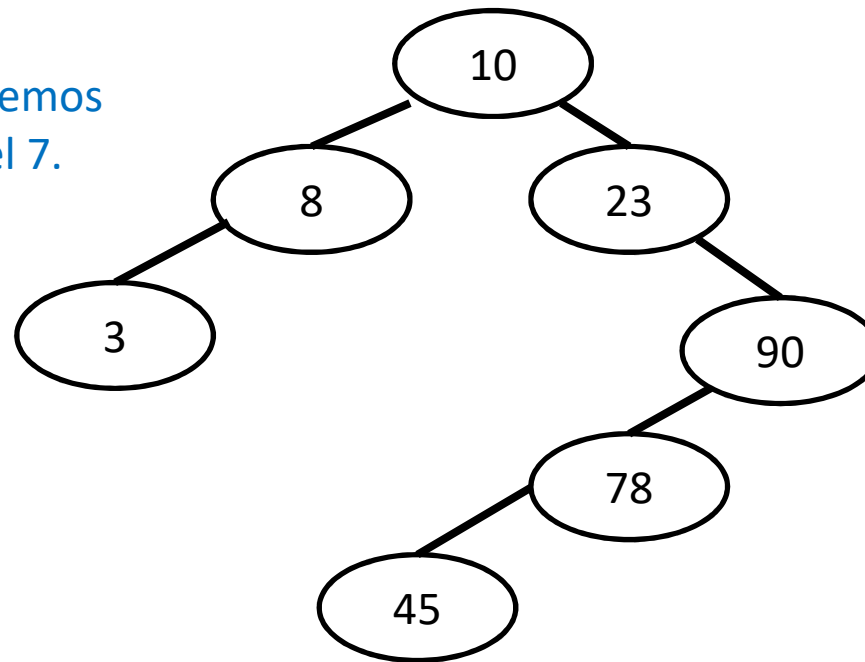


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓

Ahora queremos
insertar el 7.

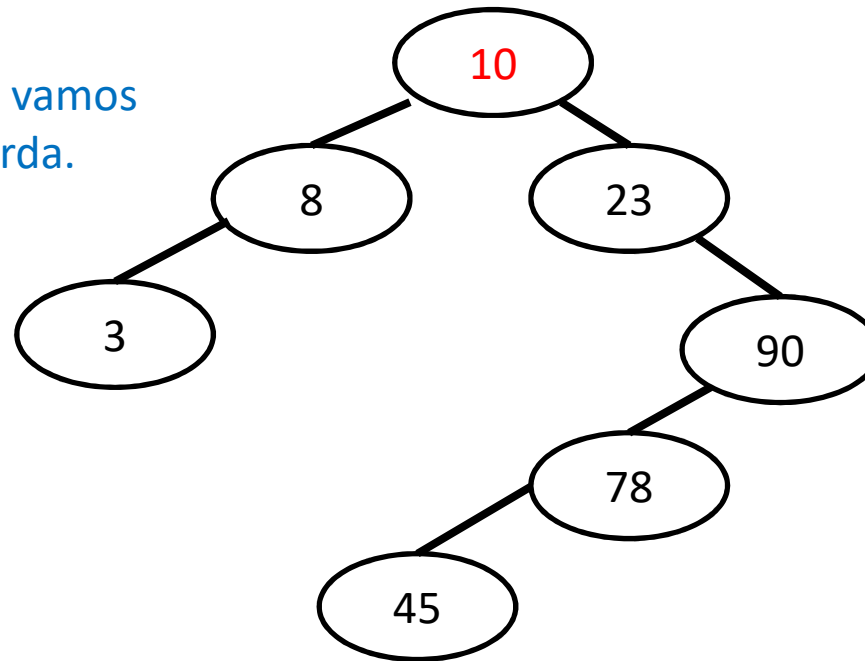


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $7 < 10$, vamos
a la izquierda.

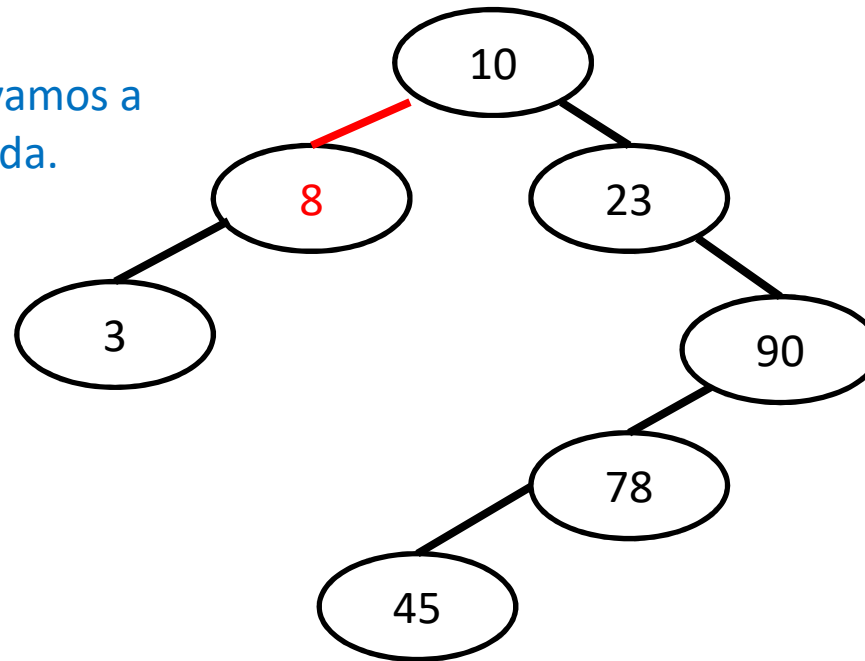


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $7 < 8$, vamos a la izquierda.

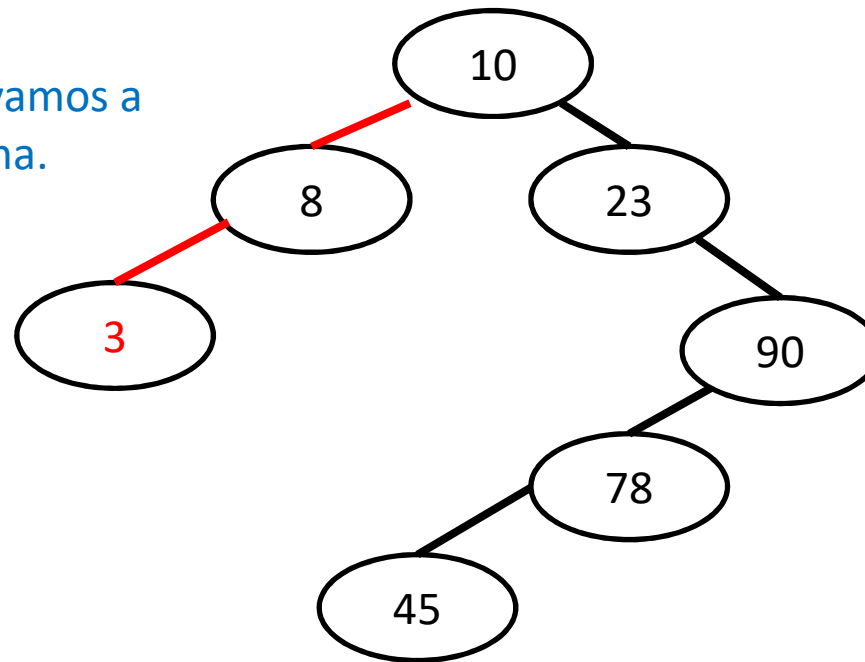


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $7 > 3$, vamos a la derecha.

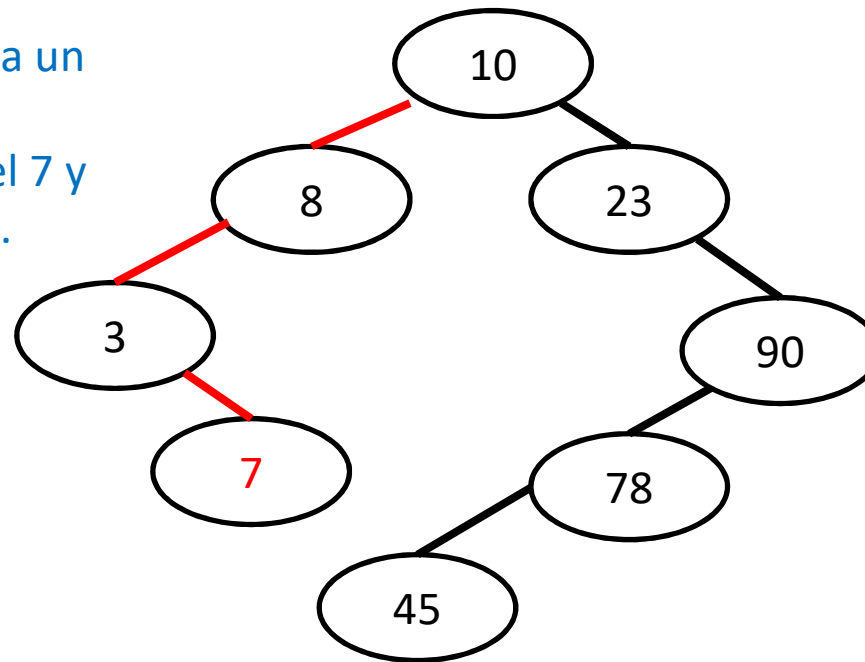


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

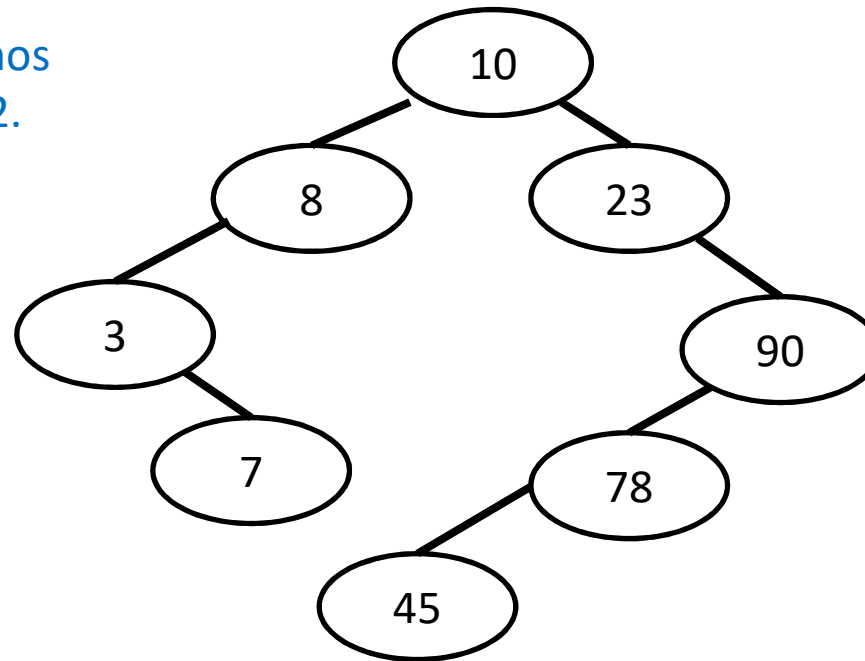
Como llegamos a un
nodo vacío,
insertamos allí el 7 y
terminamos.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

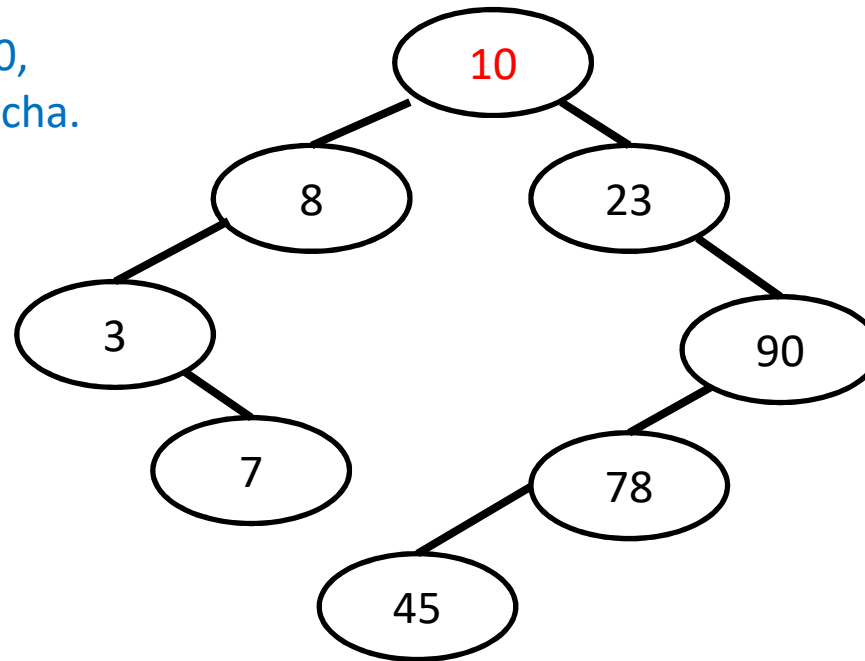
Ahora queremos
insertar el 12.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $12 > 10$,
vamos a la derecha.

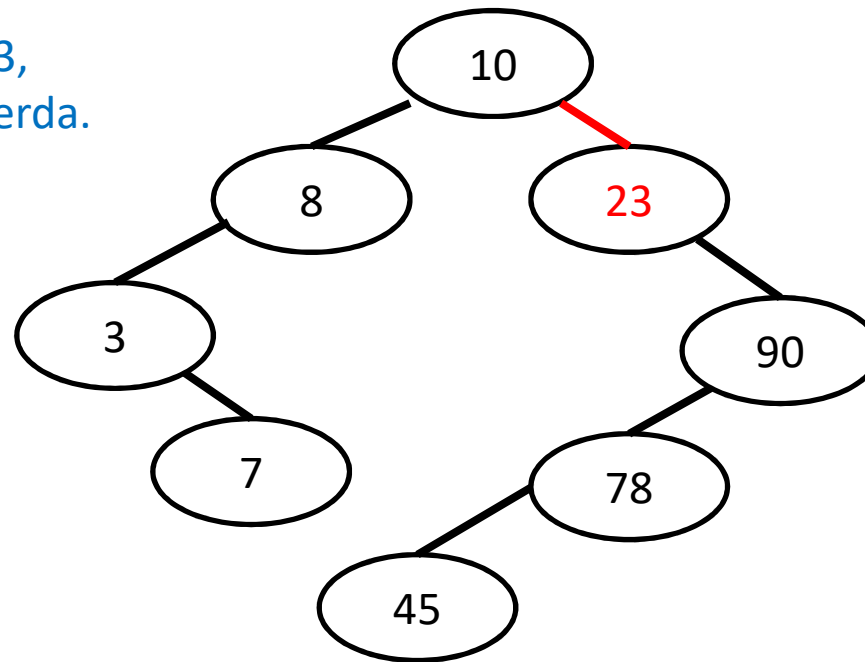


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $12 < 23$,
vamos a la izquierda.

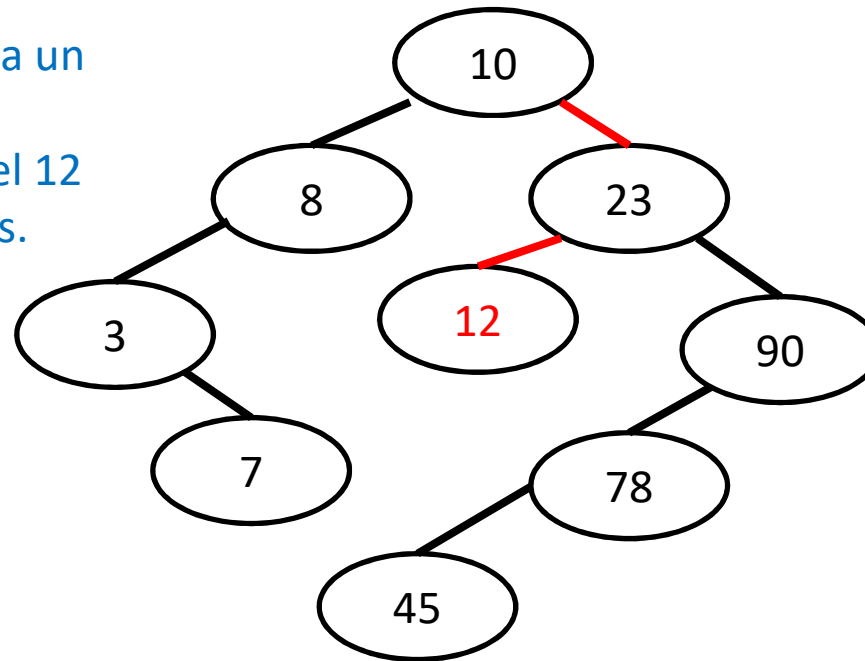


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

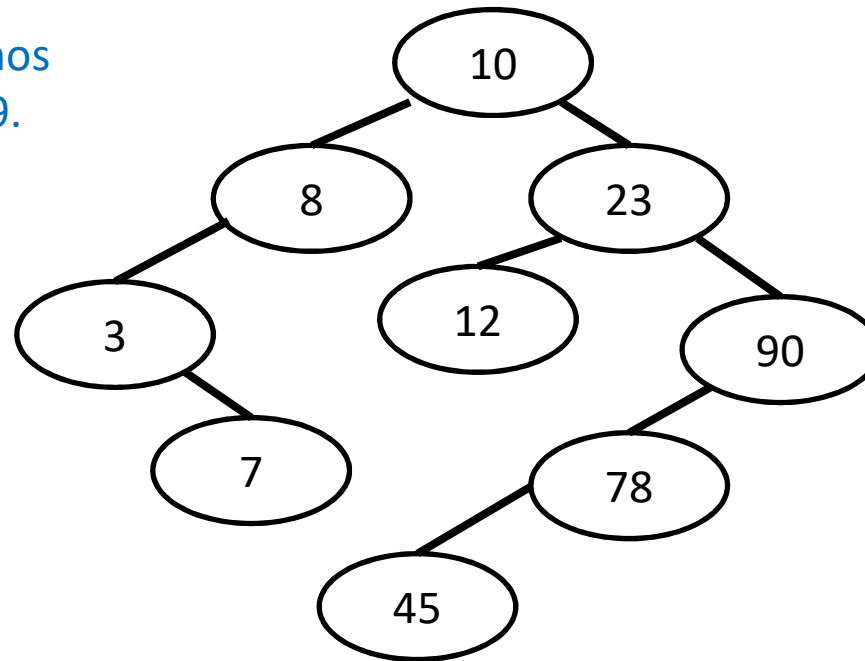
Como llegamos a un
nodo vacío,
insertamos allí el 12
y terminamos.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

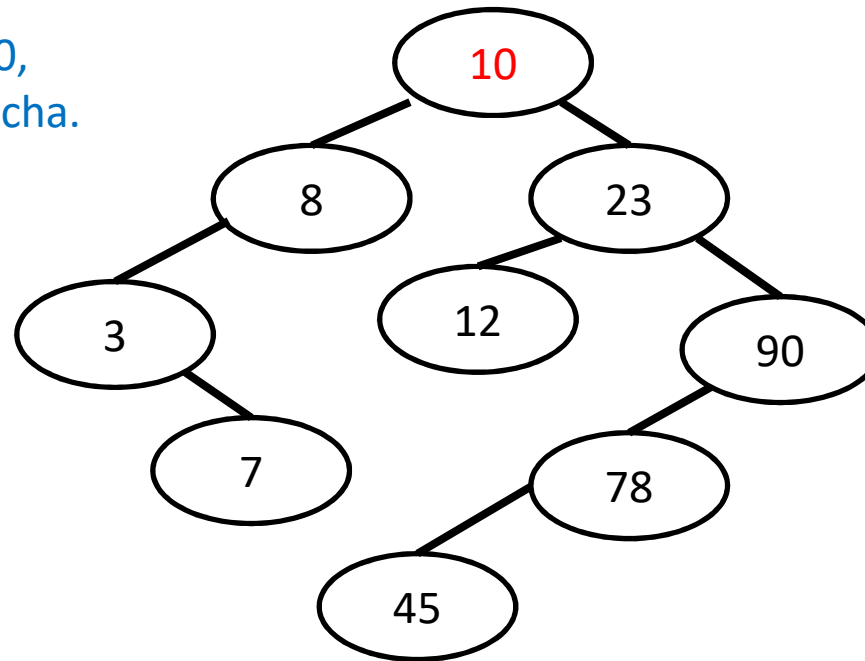
Ahora queremos
insertar el 89.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

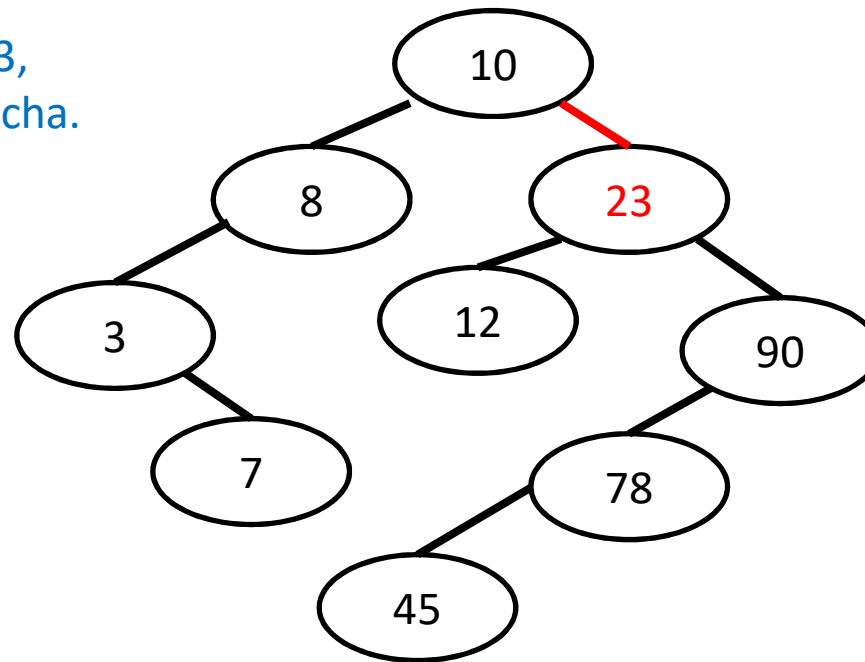
Como $89 > 10$,
vamos a la derecha.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

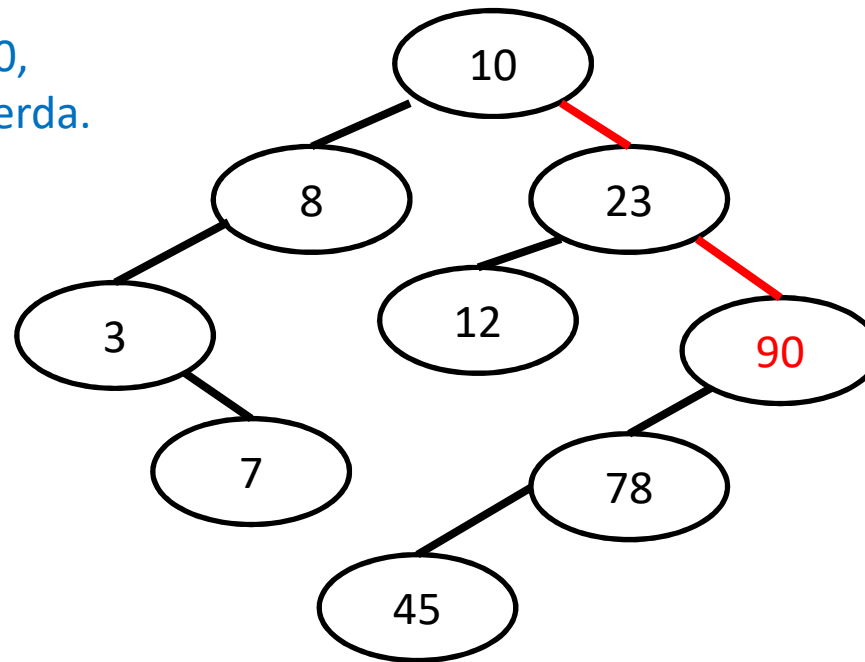
Como $89 > 23$,
vamos a la derecha.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $89 < 90$,
vamos a la izquierda.

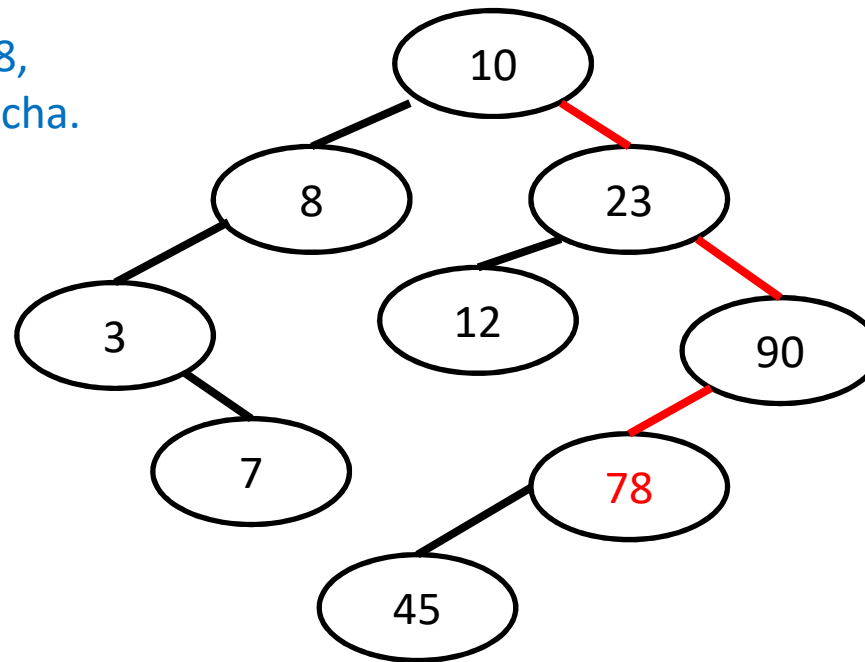


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

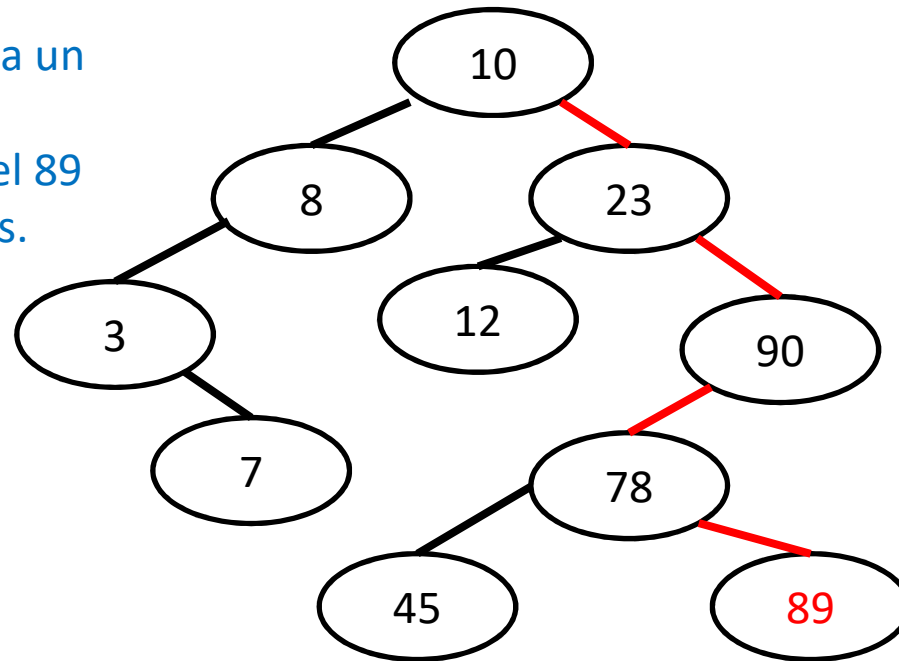
Como $89 > 78$,
vamos a la derecha.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

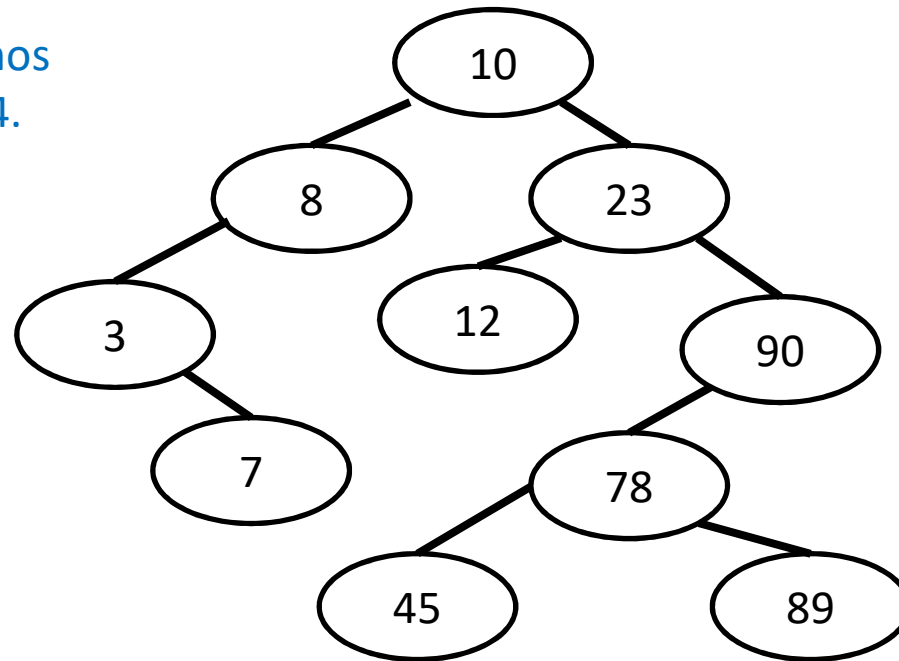
Como llegamos a un
nodo vacío,
insertamos allí el 89
y terminamos.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

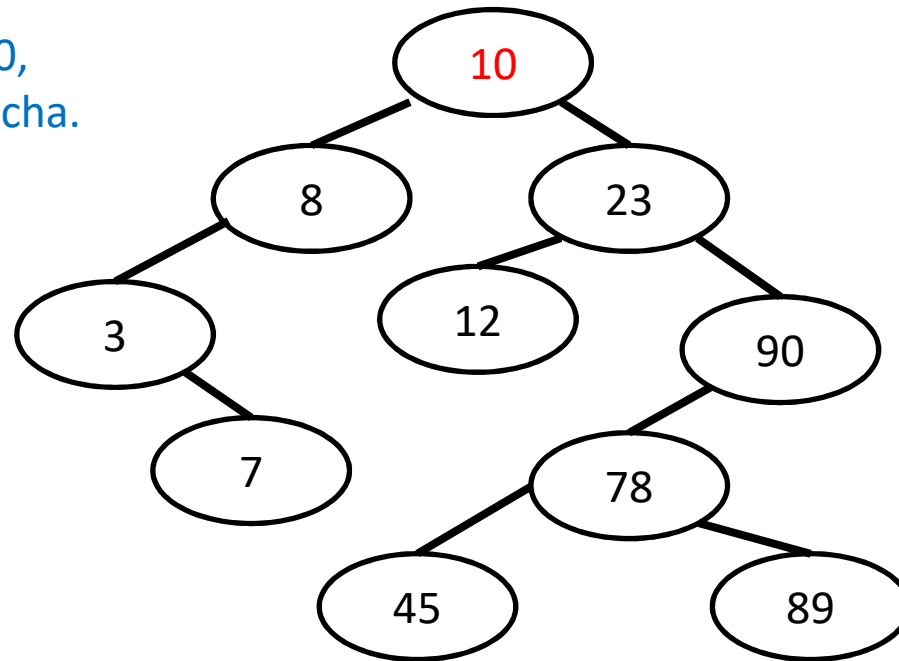
Ahora queremos
insertar el 44.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

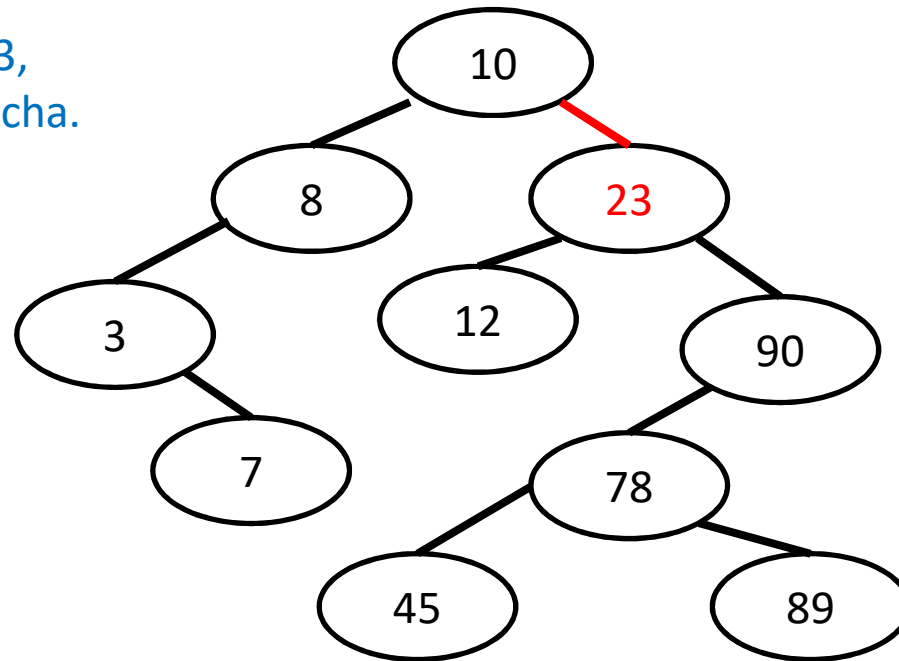
Como $44 > 10$,
vamos a la derecha.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $44 > 23$,
vamos a la derecha.

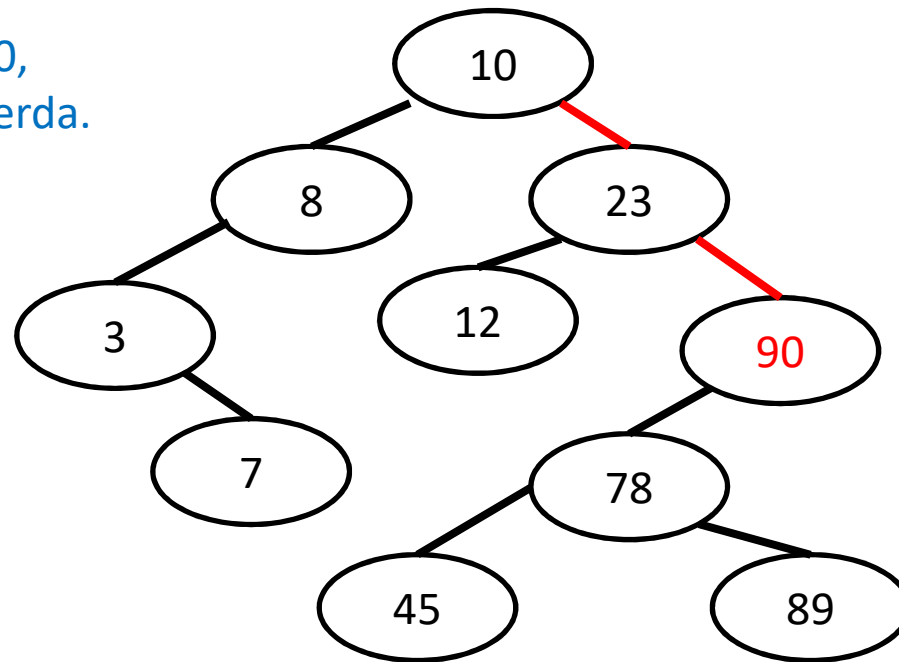


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $44 < 90$,
vamos a la izquierda.

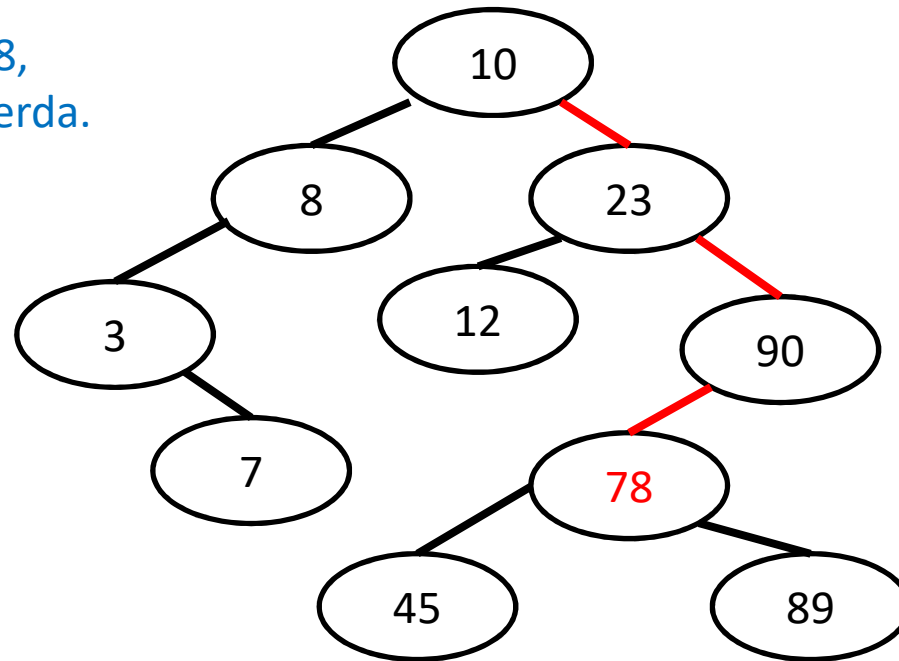


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

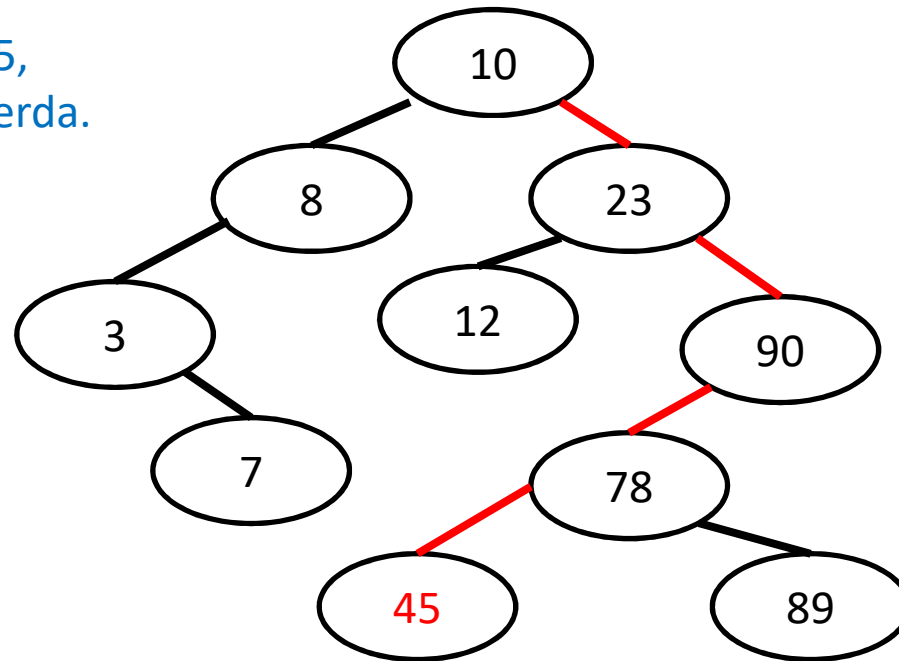
Como $44 < 78$,
vamos a la izquierda.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

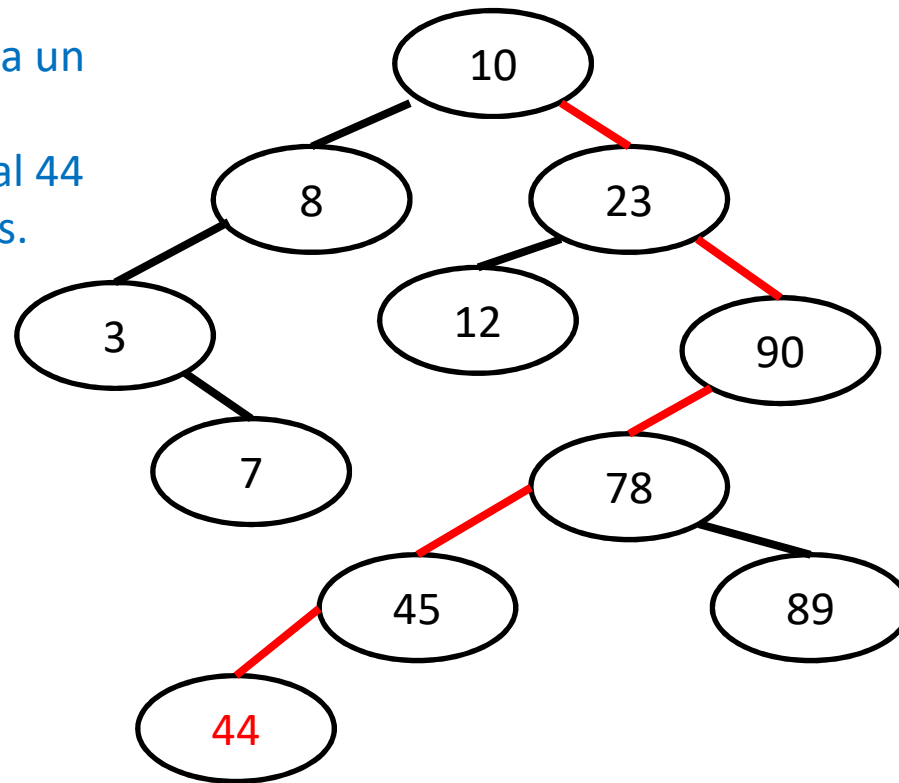
Como $44 < 45$,
vamos a la izquierda.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como llegamos a un
nodo vacío,
insertamos ahí al 44
y terminamos.

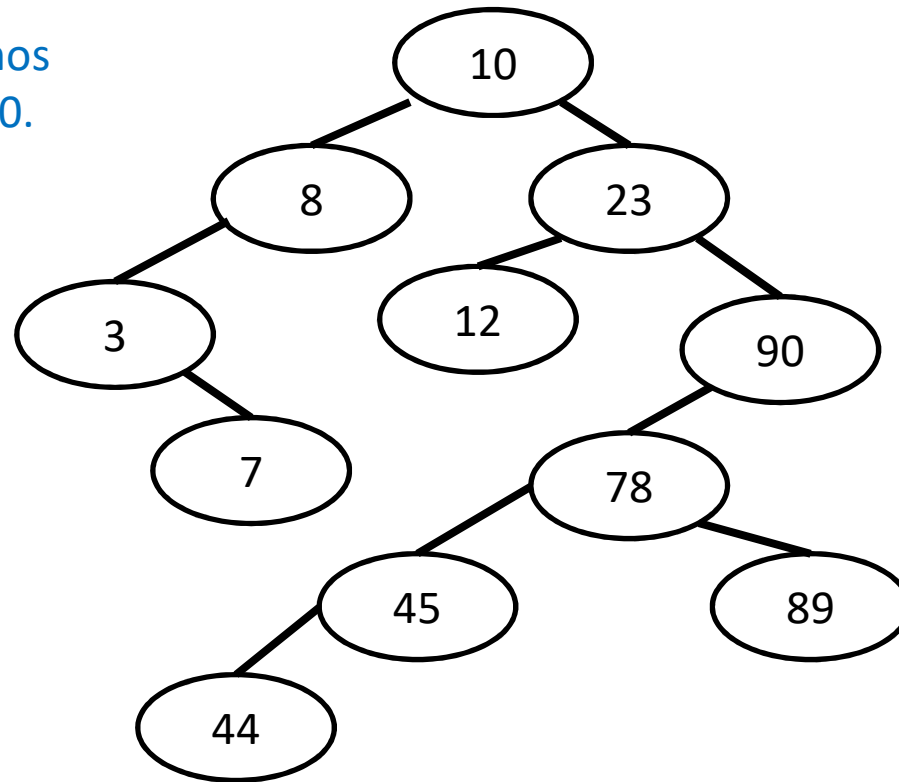


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

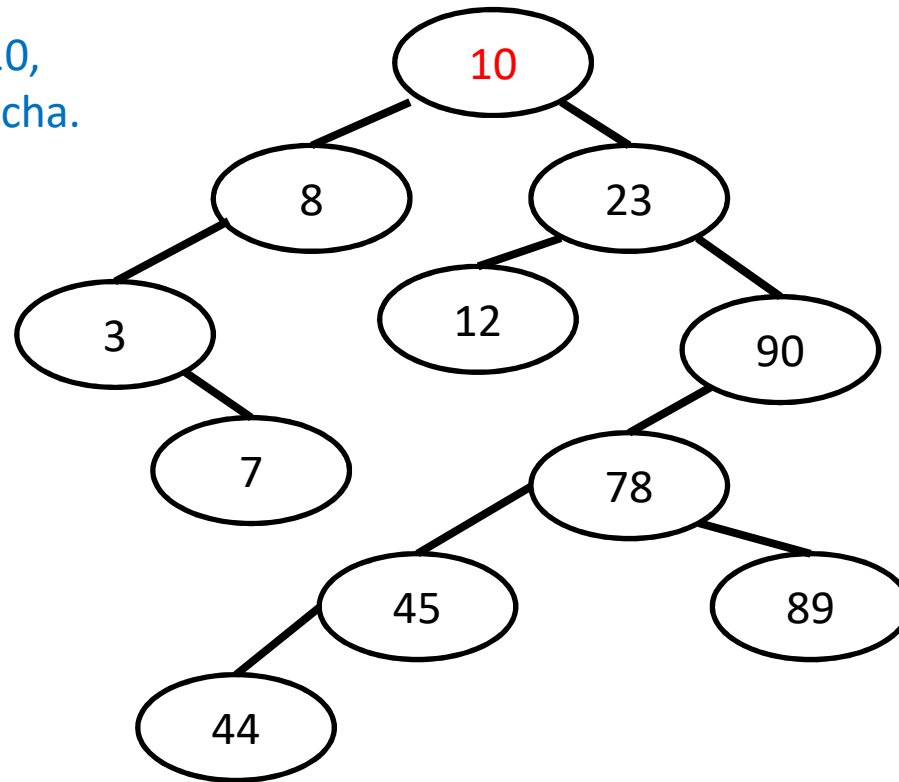
Ahora queremos
insertar el 100.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $100 > 10$,
vamos a la derecha.

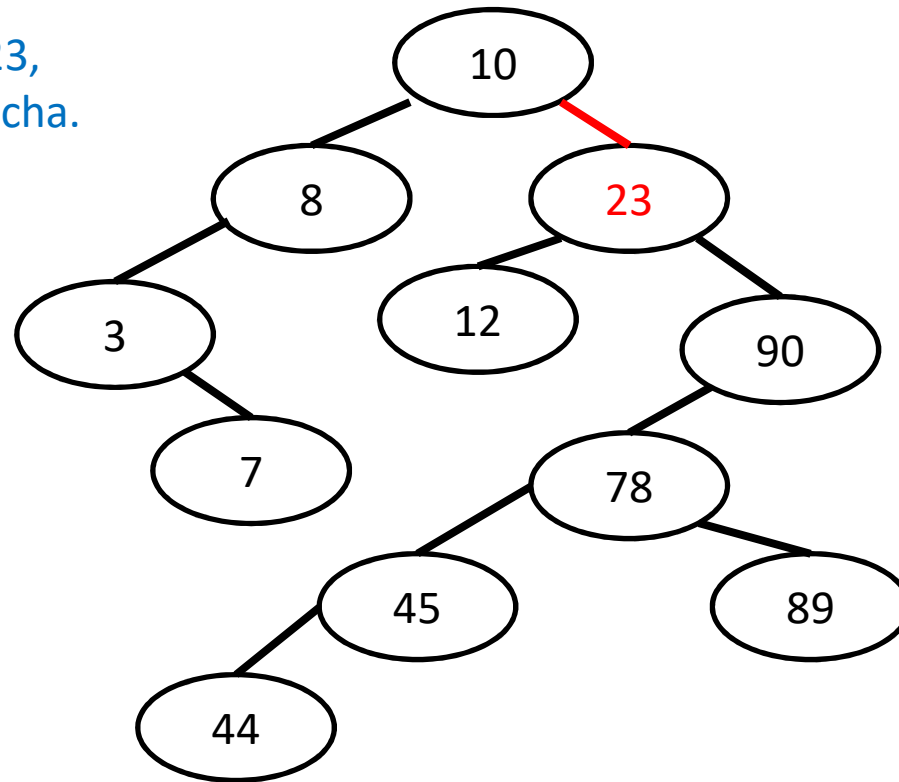


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

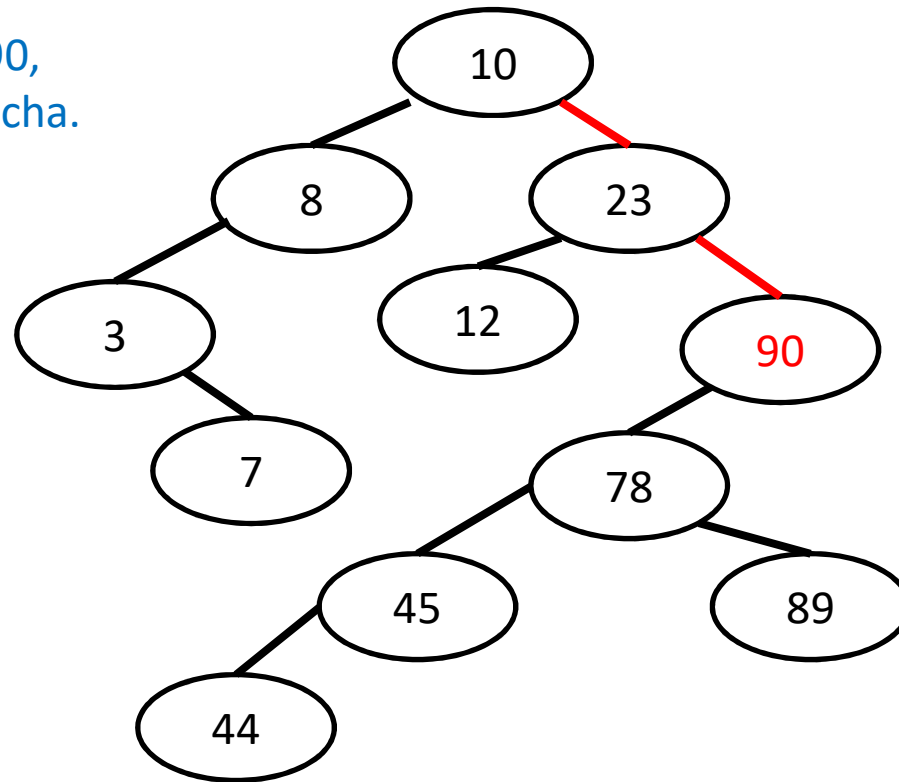
Como $100 > 23$,
vamos a la derecha.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $100 > 90$,
vamos a la derecha.

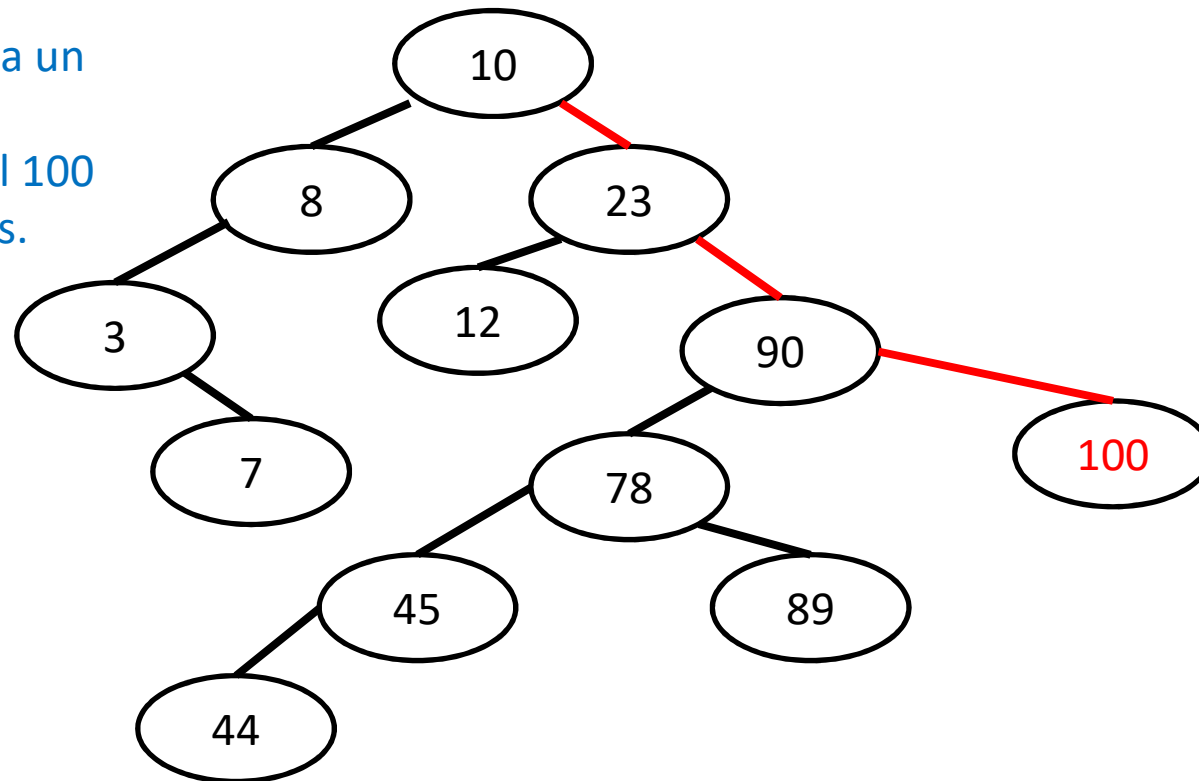


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

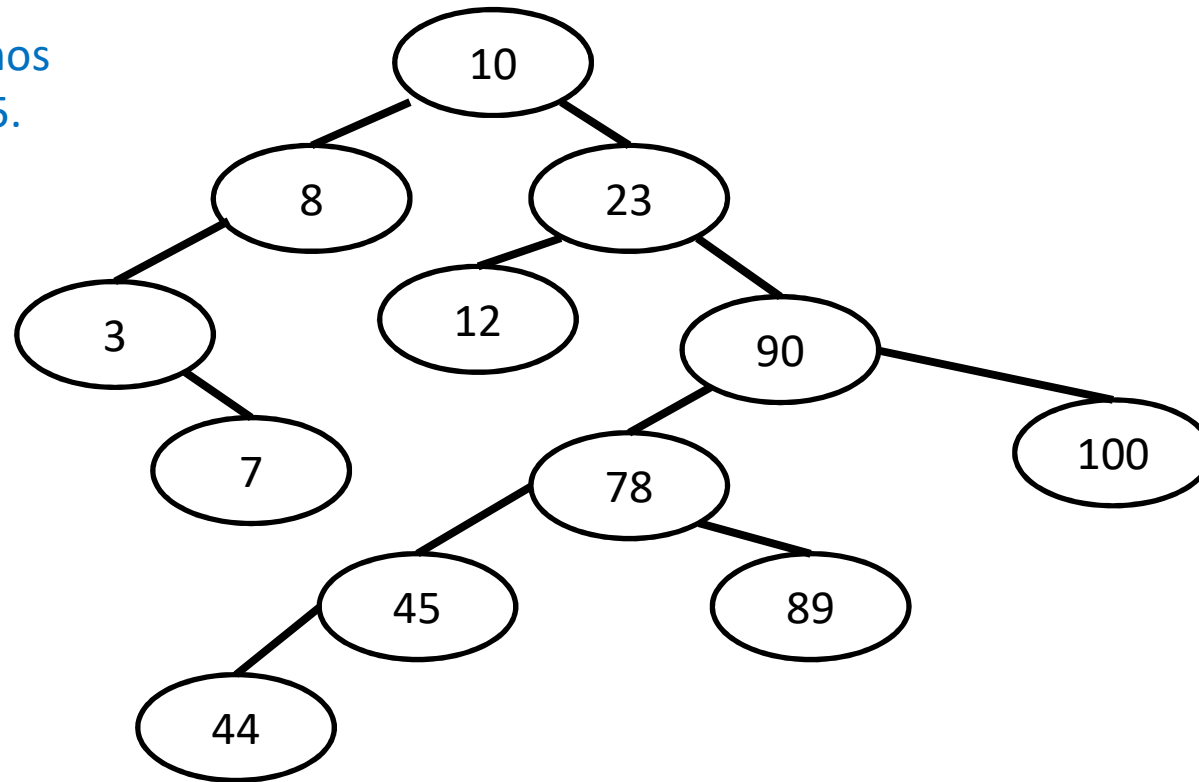
Como llegamos a un
nodo vacío,
insertamos allí al 100
y terminamos.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

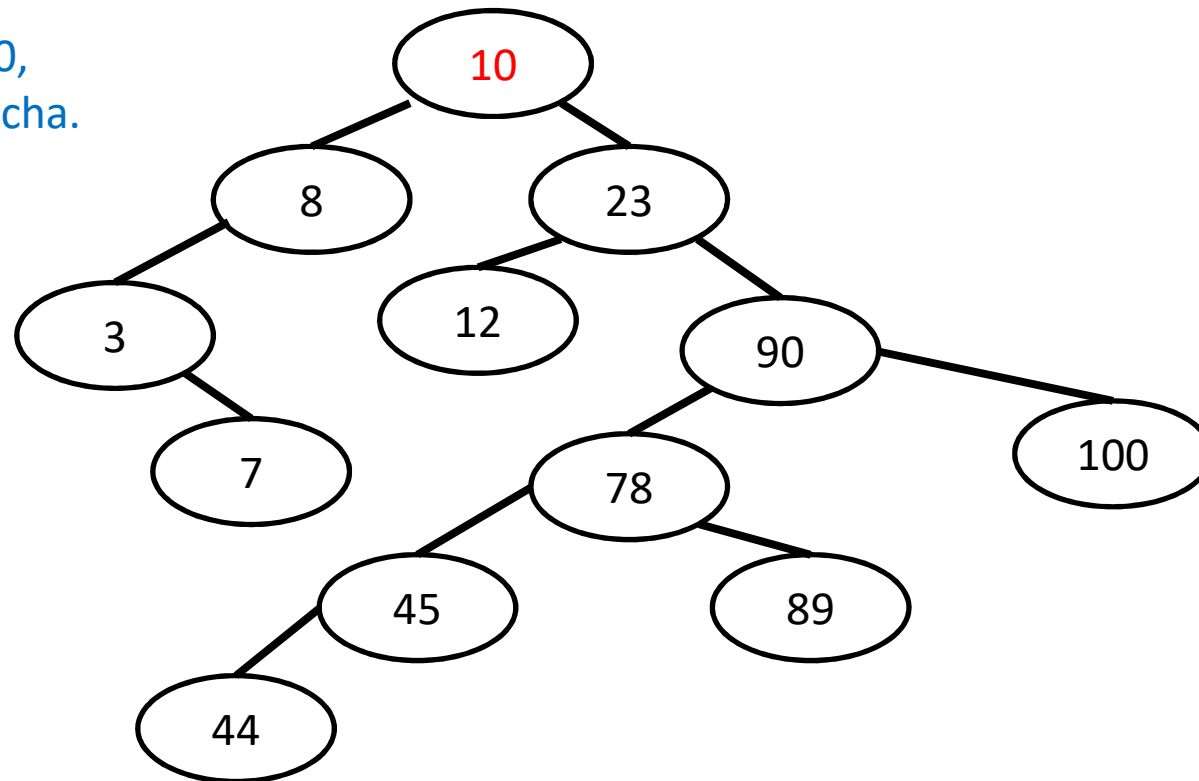
Ahora queremos
insertar el 95.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $95 > 10$,
vamos a la derecha.

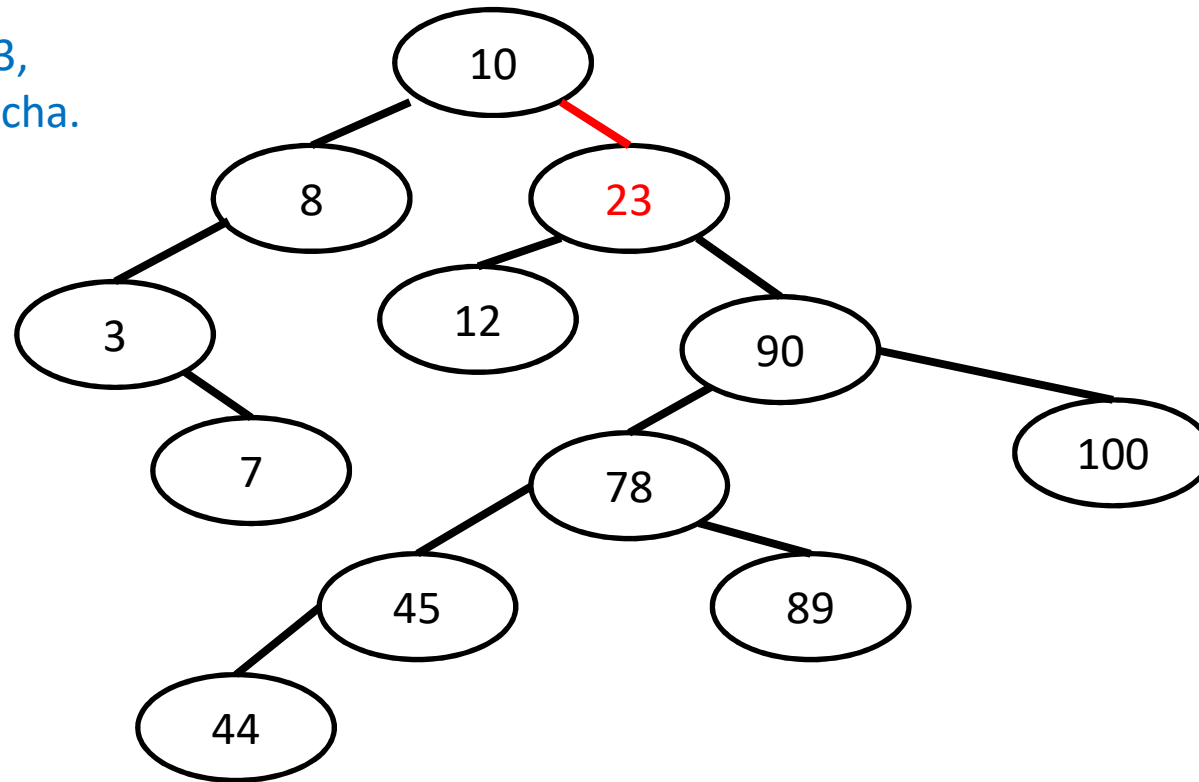


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $95 > 23$,
vamos a la derecha.

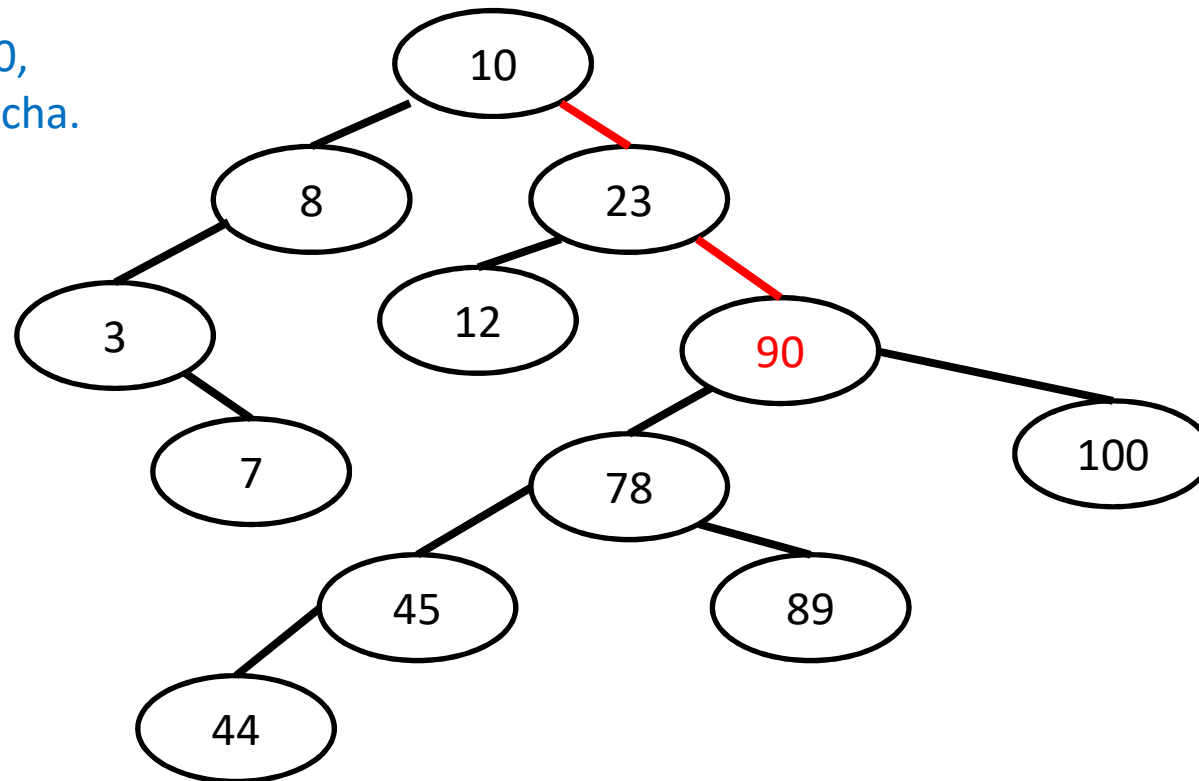


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $95 > 90$,
vamos a la derecha.

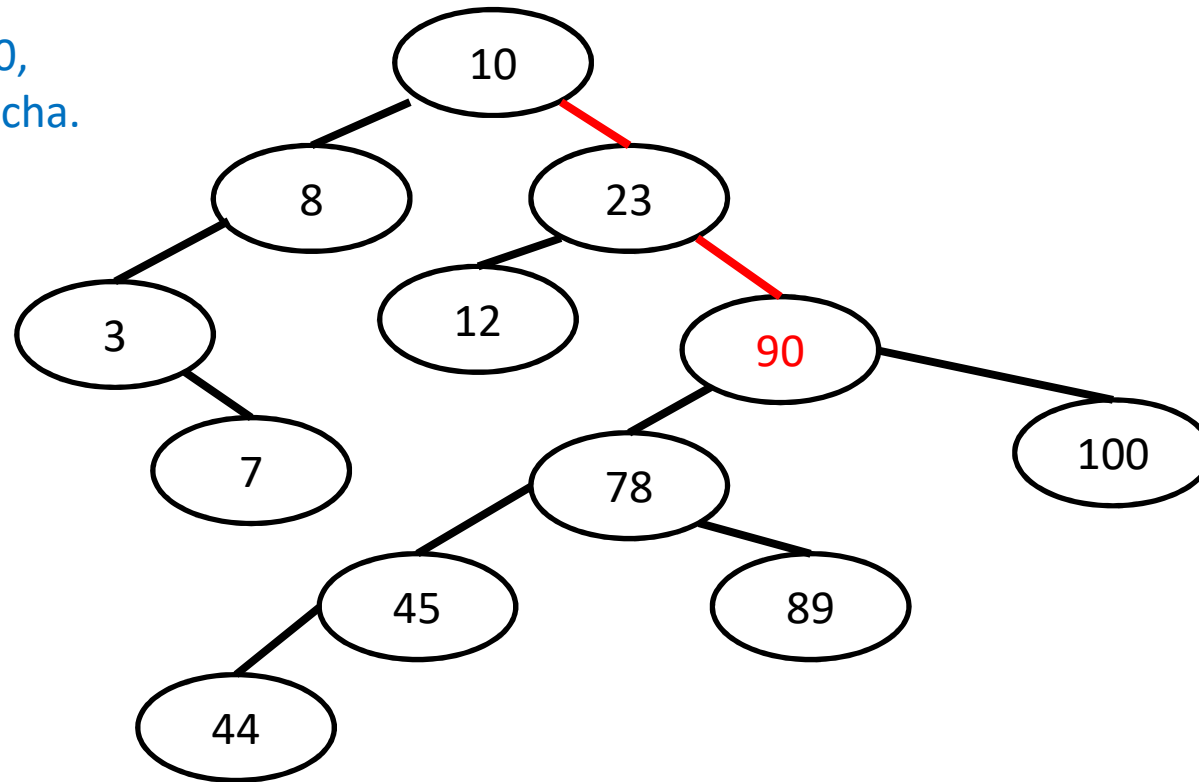


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $95 > 90$,
vamos a la derecha.

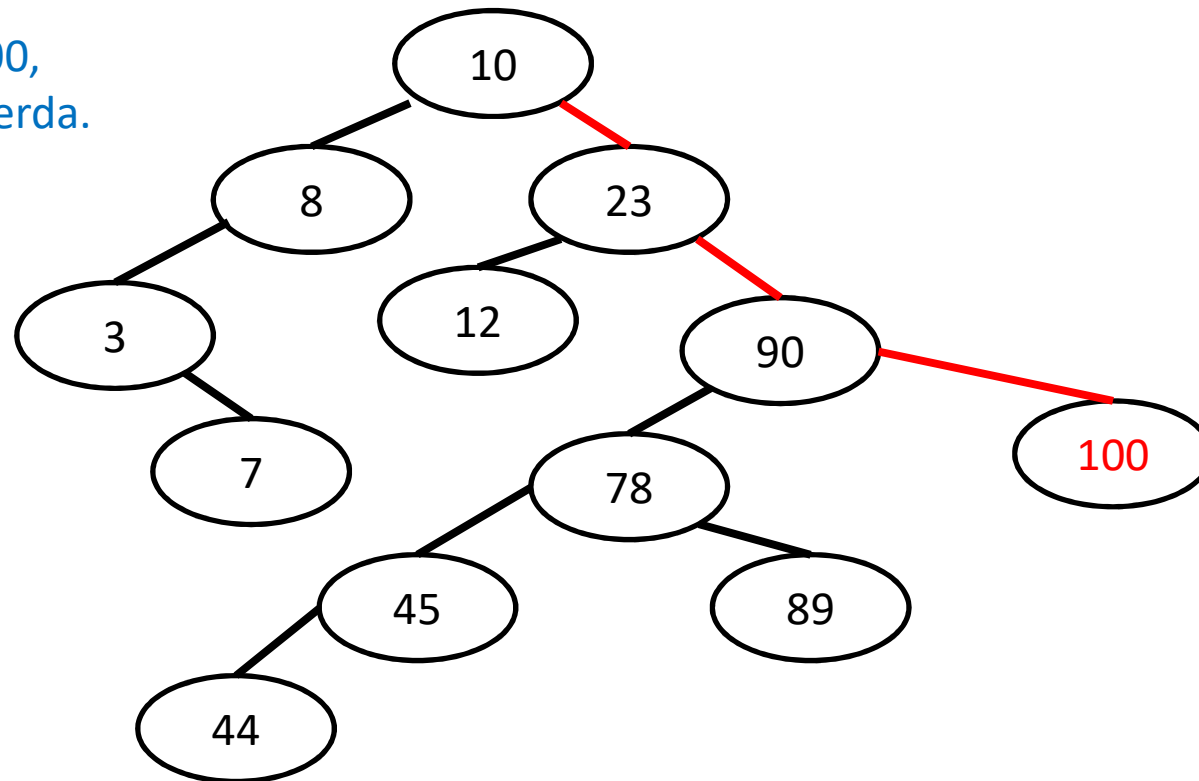


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $95 < 100$,
vamos a la izquierda.

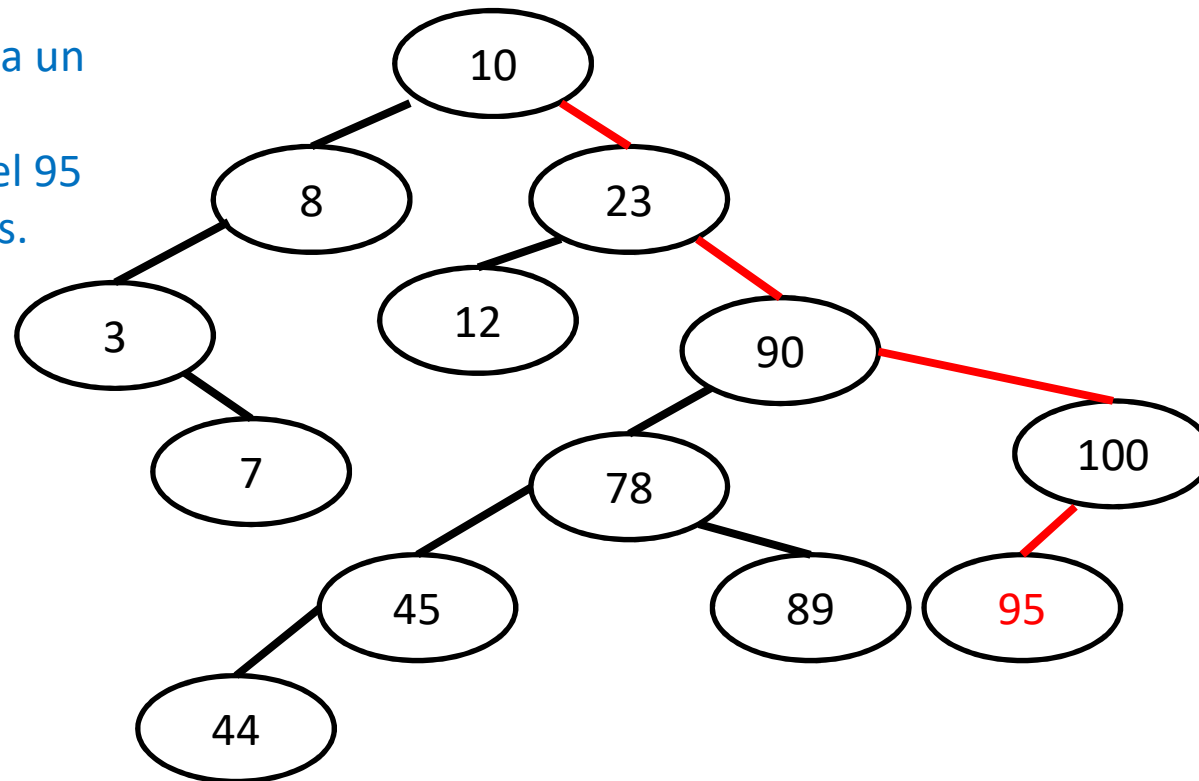


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como llegamos a un
nodo vacío,
insertamos allí el 95
y terminamos.

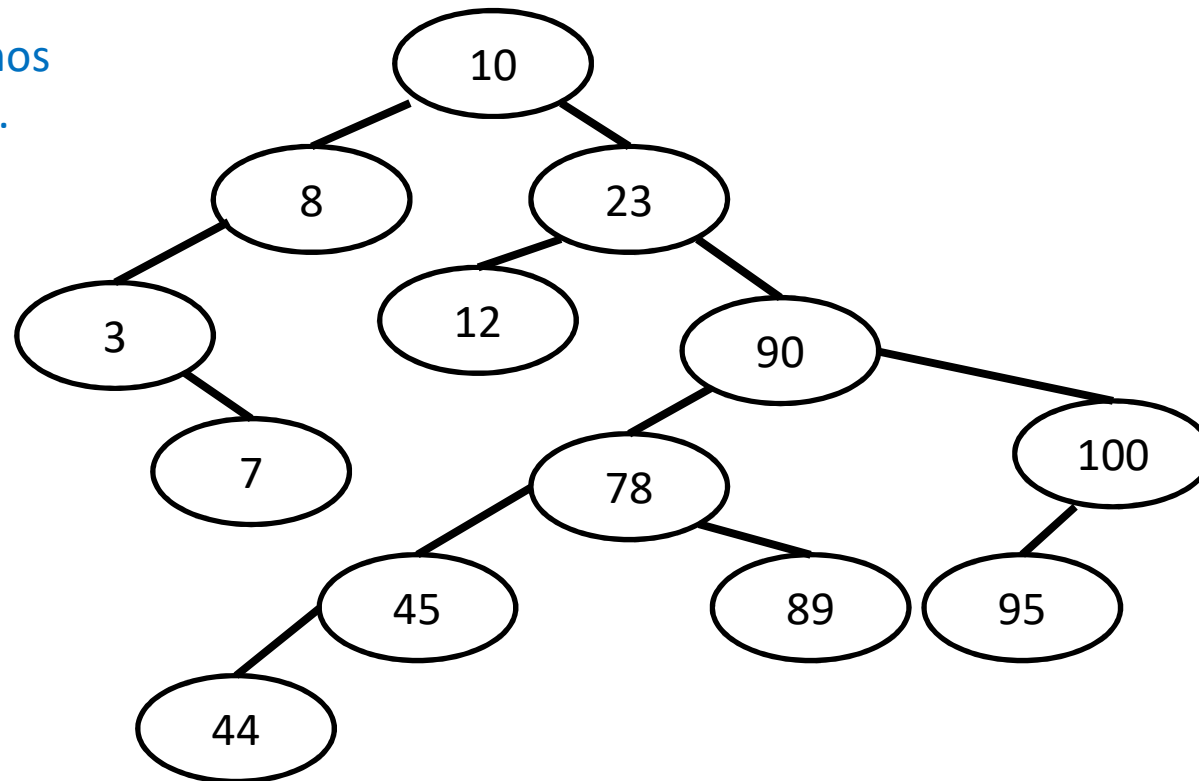


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

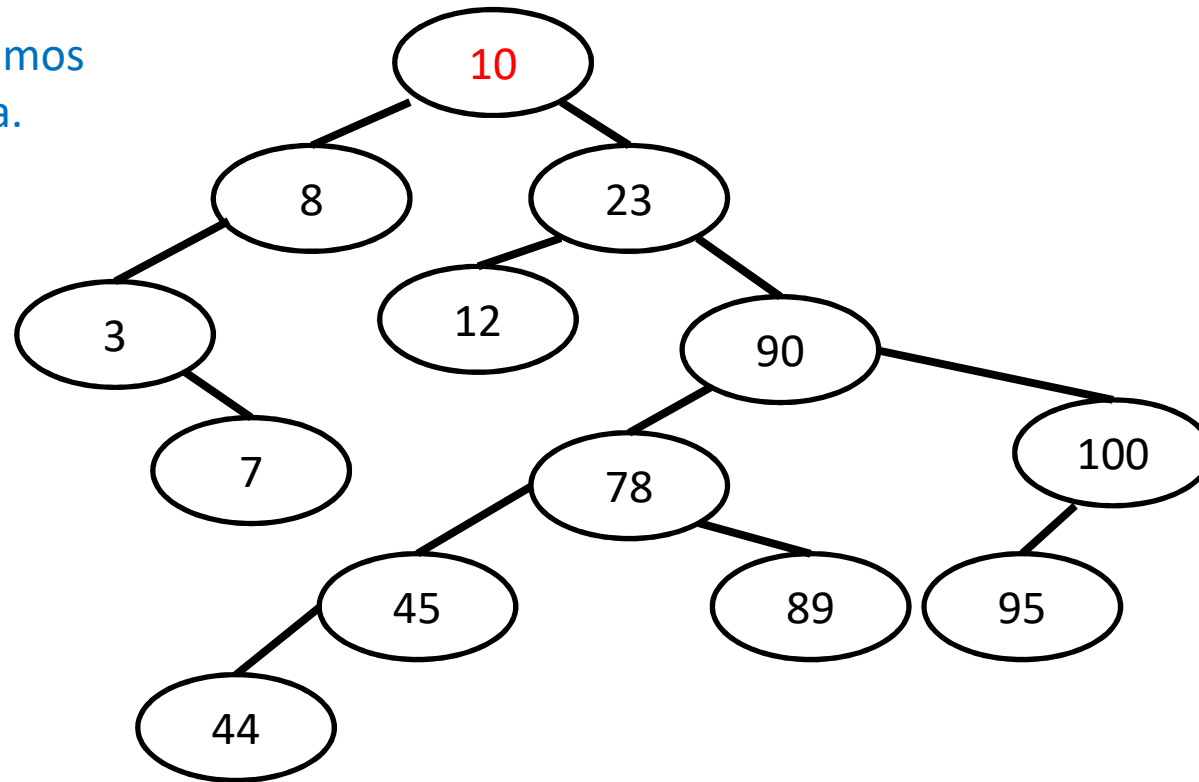
Ahora queremos
insertar el 4.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $4 < 10$, vamos
a la izquierda.

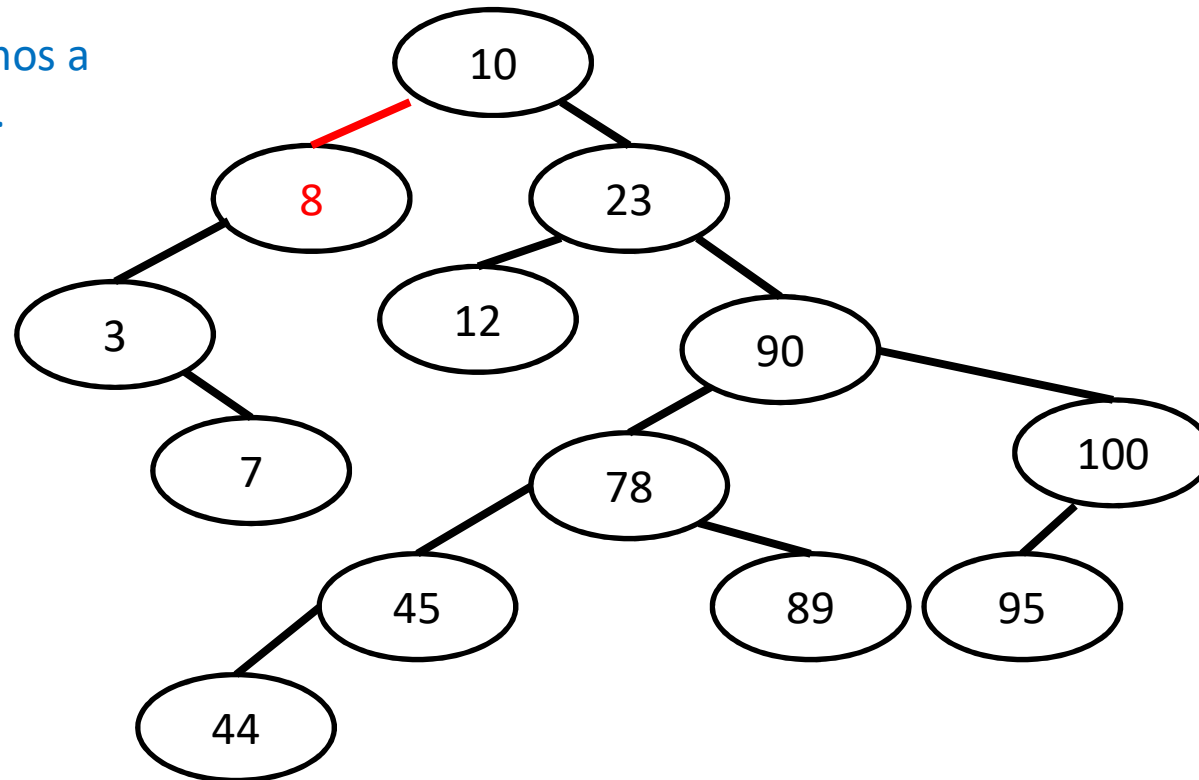


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $4 < 8$, vamos a la izquierda.

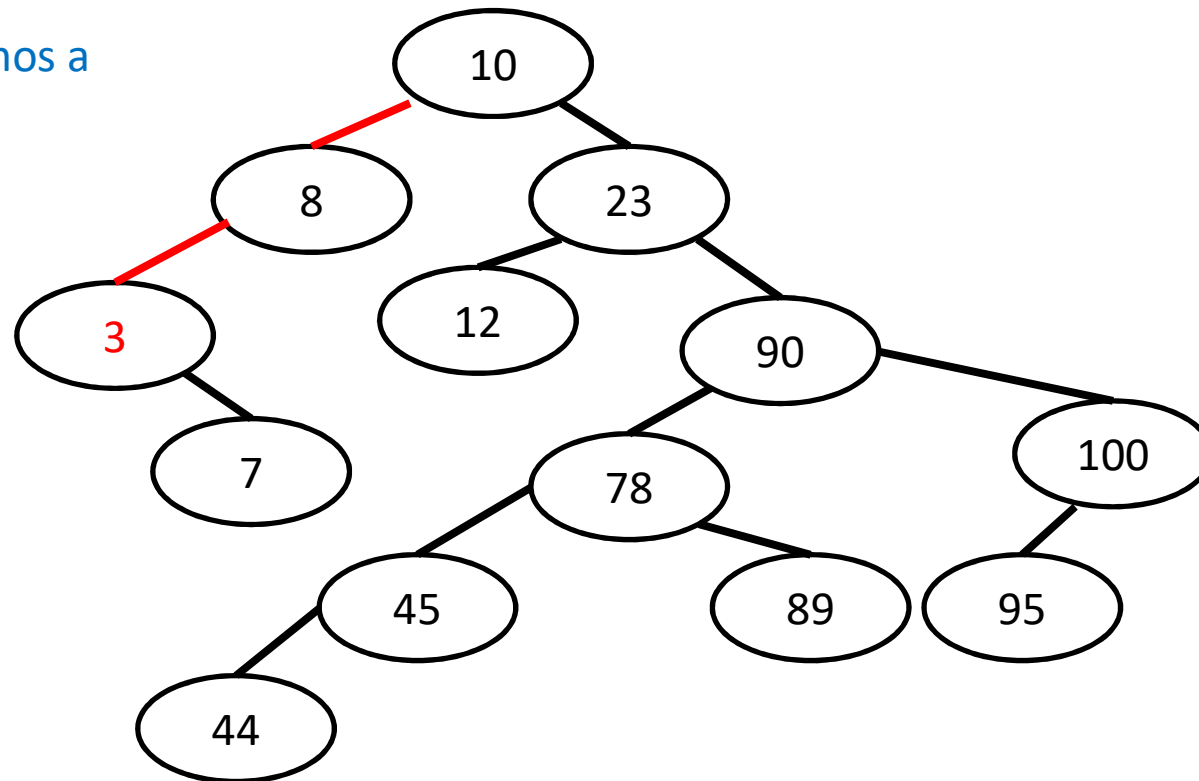


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $4 > 3$, vamos a la derecha.

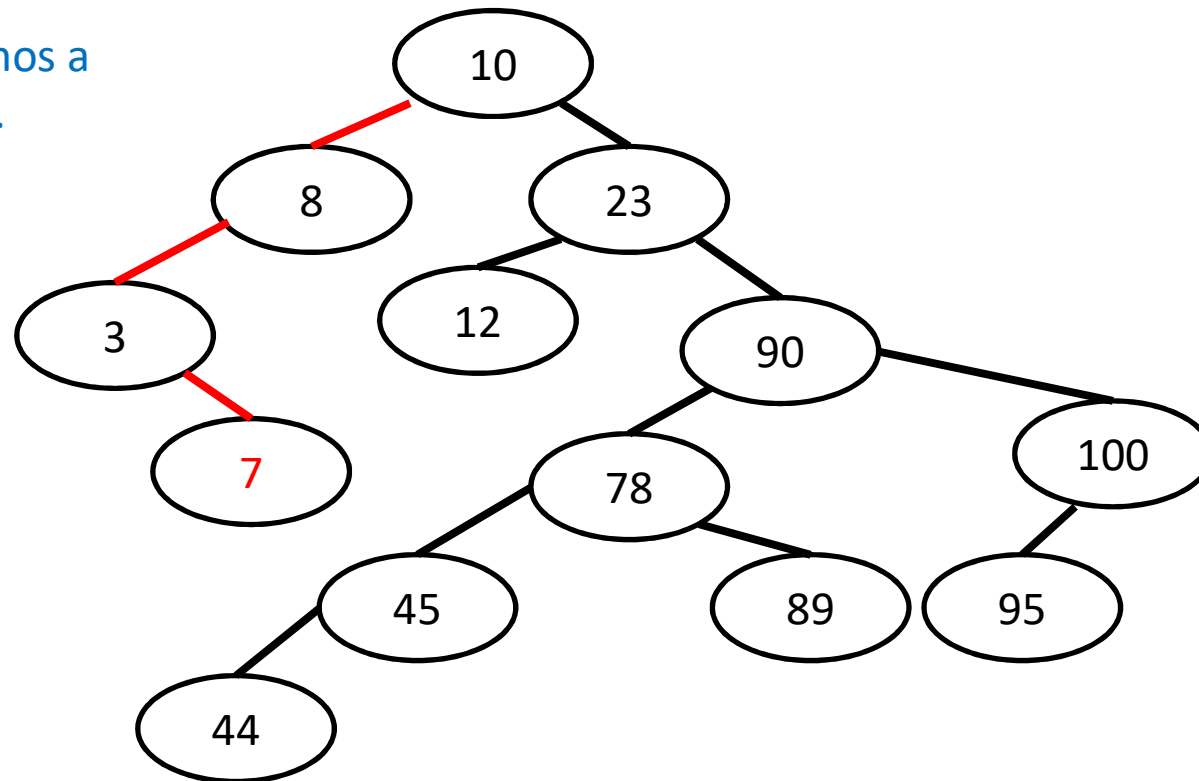


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como $4 < 7$, vamos a la izquierda.

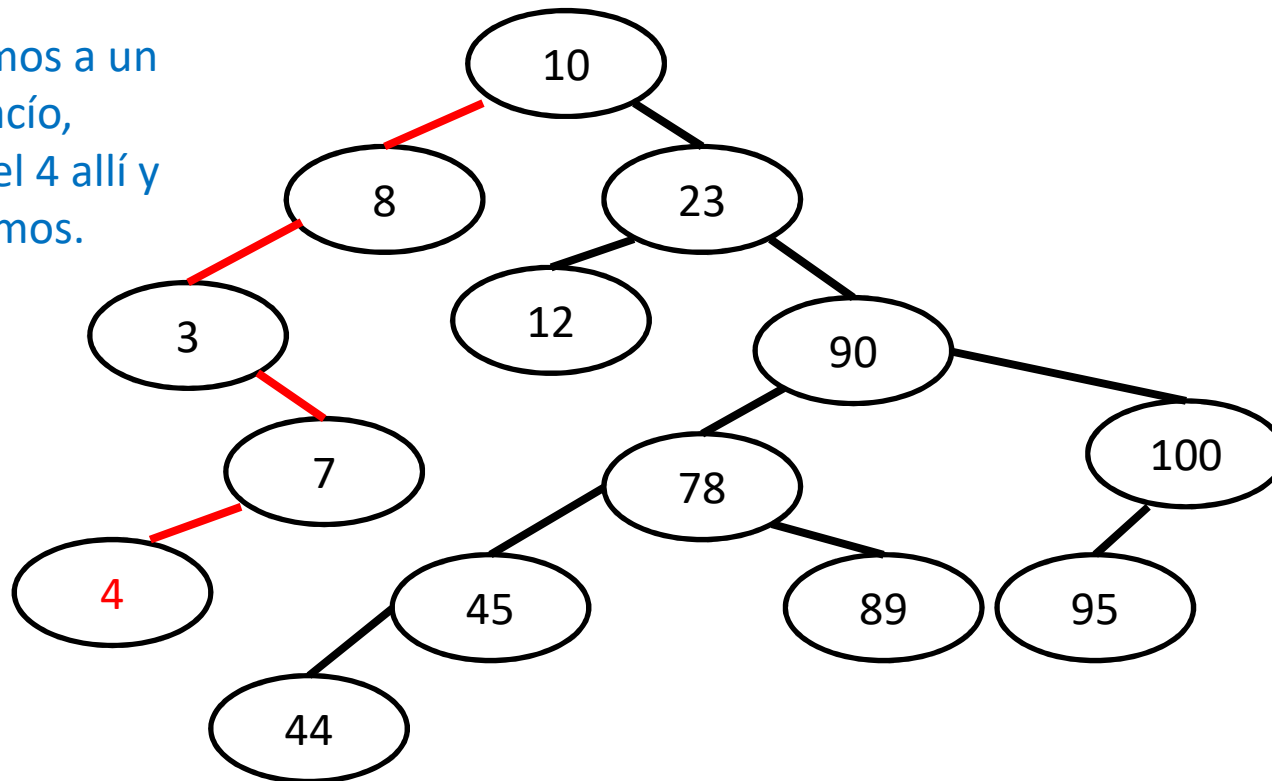


- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4

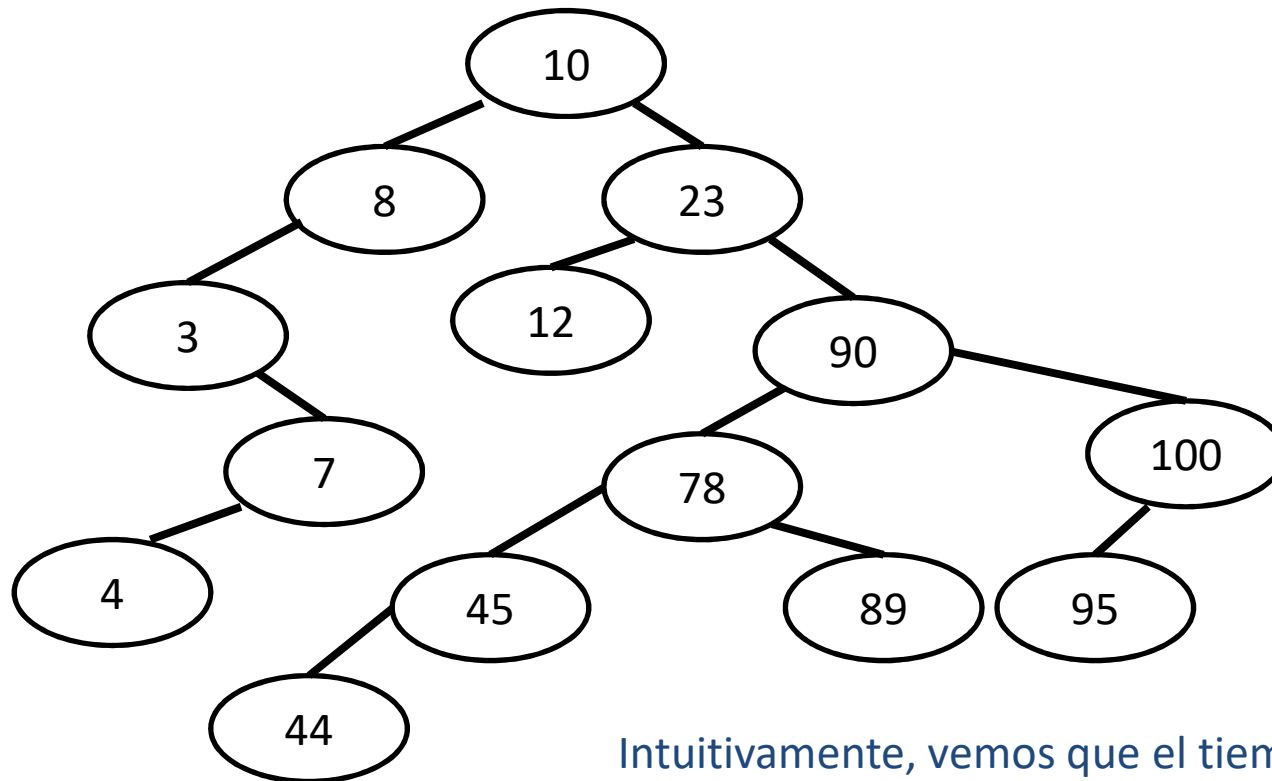
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Como llegamos a un
nodo vacío,
insertamos el 4 allí y
terminamos.



- Ejemplo 1: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

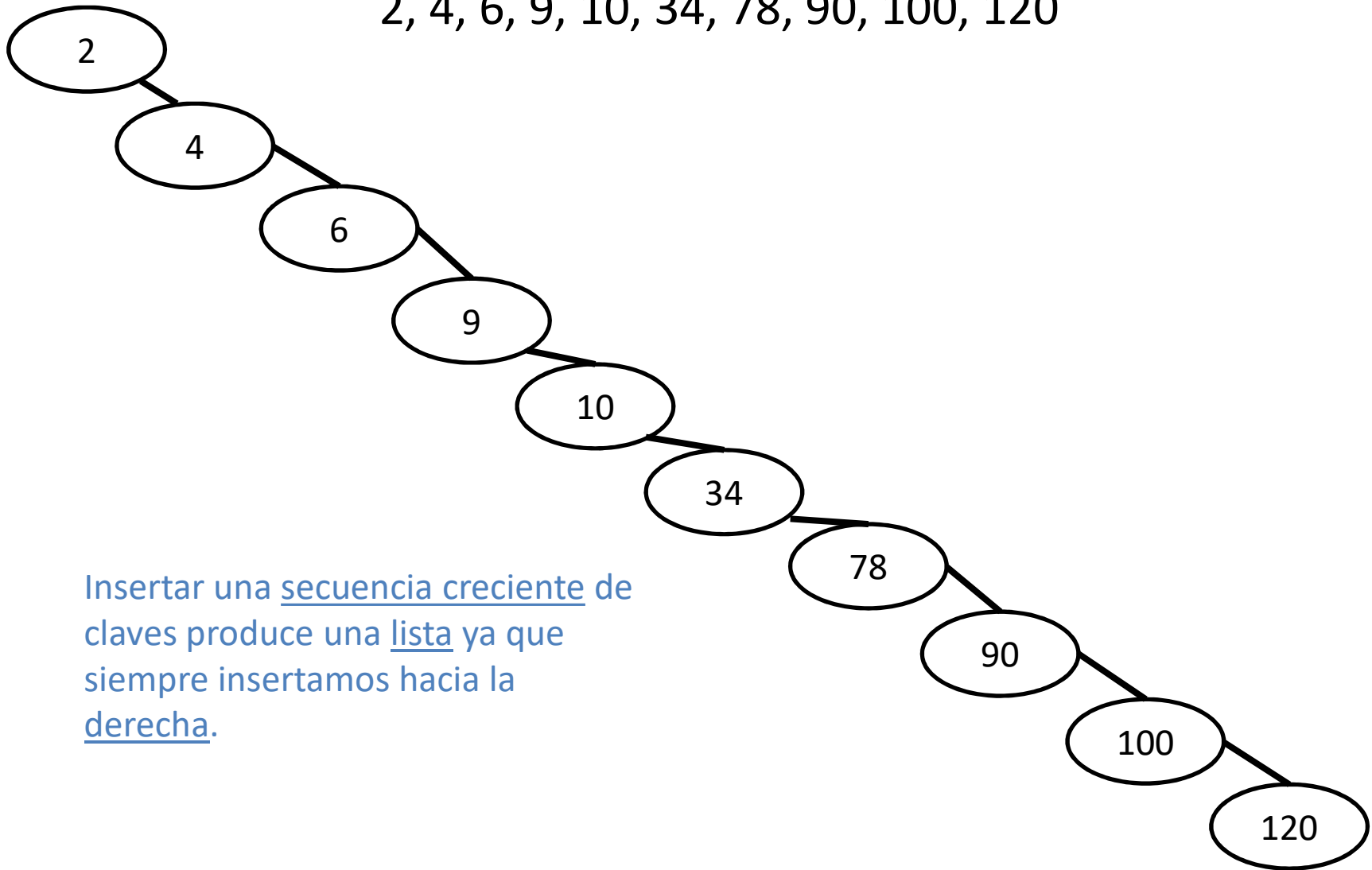
10, 23, 8, 90, 78, 45, 3, 7, 12, 89, 44, 100, 95, 4



Intuitivamente, vemos que el tiempo de ejecución de la inserción es lineal en el largo del camino de la raíz al nodo donde se inserta el rótulo nuevo.

Ejemplo 2: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

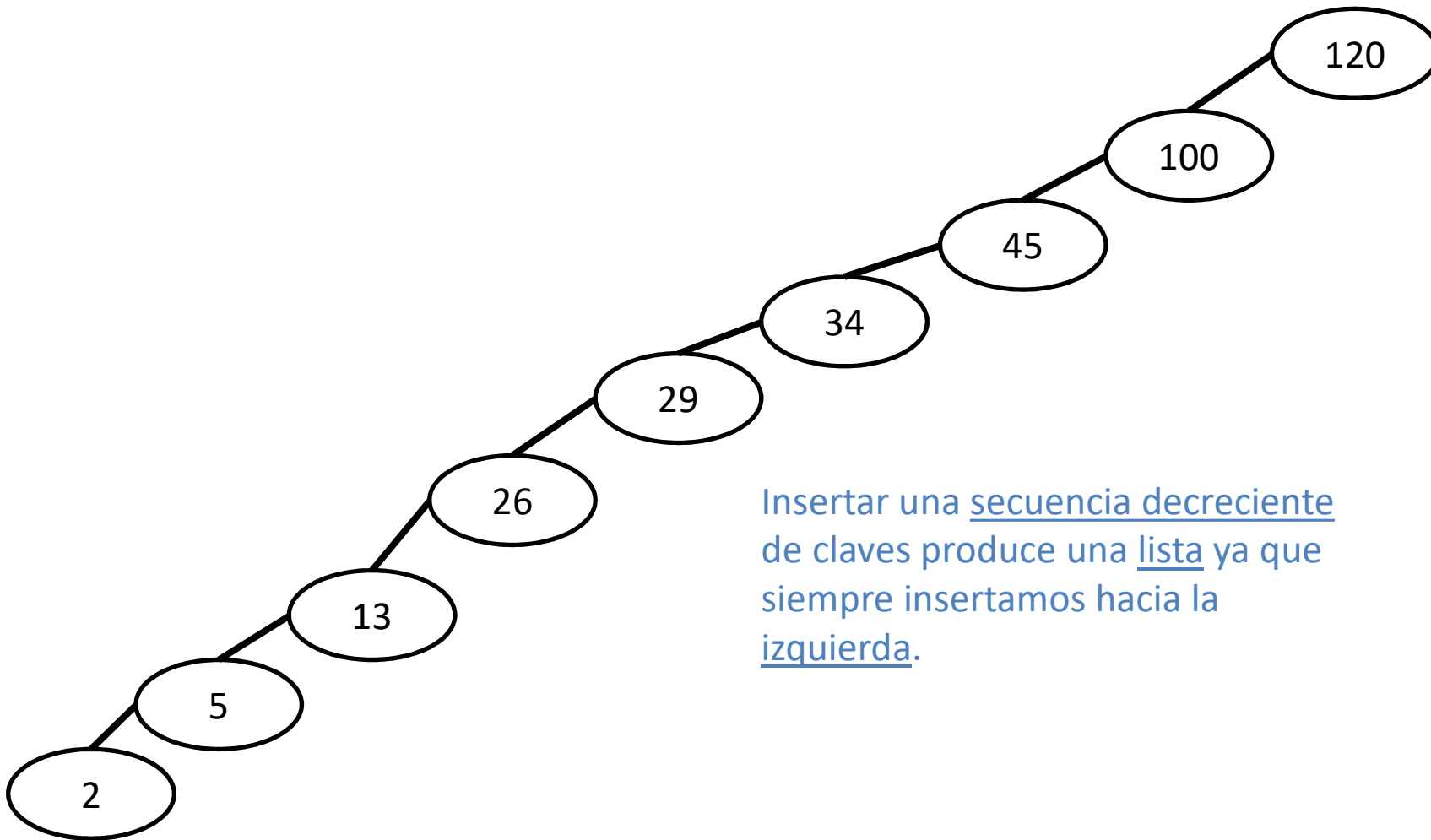
2, 4, 6, 9, 10, 34, 78, 90, 100, 120



Insertar una secuencia creciente de claves produce una lista ya que siempre insertamos hacia la derecha.

Ejemplo 3: Dado un ABB vacío, realizar la siguiente secuencia de inserciones:

120, 100, 45, 34, 29, 26, 16, 13, 5, 2



Insertar una secuencia decreciente de claves produce una lista ya que siempre insertamos hacia la izquierda.

Análisis de la complejidad temporal

- Sea h = altura del ABB y sea n = cantidad de claves del ABB.

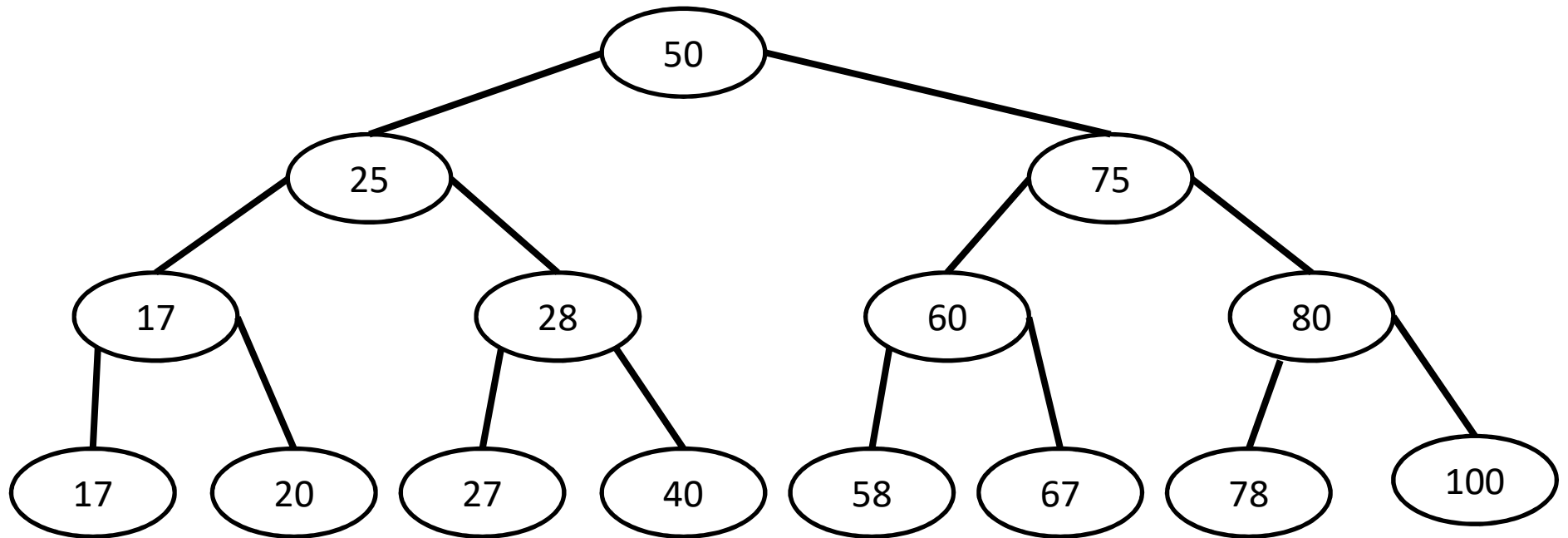
$$T(h) = \begin{cases} c_1 & \text{si } h = 0 \\ c_2 + T(h - 1) & \text{si } h > 0 \end{cases}$$

con lo cual $T(h) = O(h)$.

- La altura en el peor caso es $h = n - 1 = O(n)$ (se da cuando se hicieron inserciones de claves en forma ascendente o descendente)
- La altura en el mejor caso es $h = O(\log_2(n))$ ya que las inserciones producen un árbol lleno que tiene $n = 2^{h+1} - 1$ nodos; de ahí, despejo $h = \log_2(n + 1) - 1$.
- El árbol lleno de altura h es aquel que todos los nodos internos tienen 2 hijos y todas las hojas tienen la misma profundidad h .

Mejor árbol posible: árbol lleno de altura h

Insertar 50, 25, 75, 17, 28, 60, 80, 100, 78, 67, 58, 40, 27, 20.



Este árbol se llama lleno.

La cantidad de nodos n es igual a $2^{h+1}-1$ con h la altura del árbol.

Si despejamos $h = \log_2(n+1) - 1$.

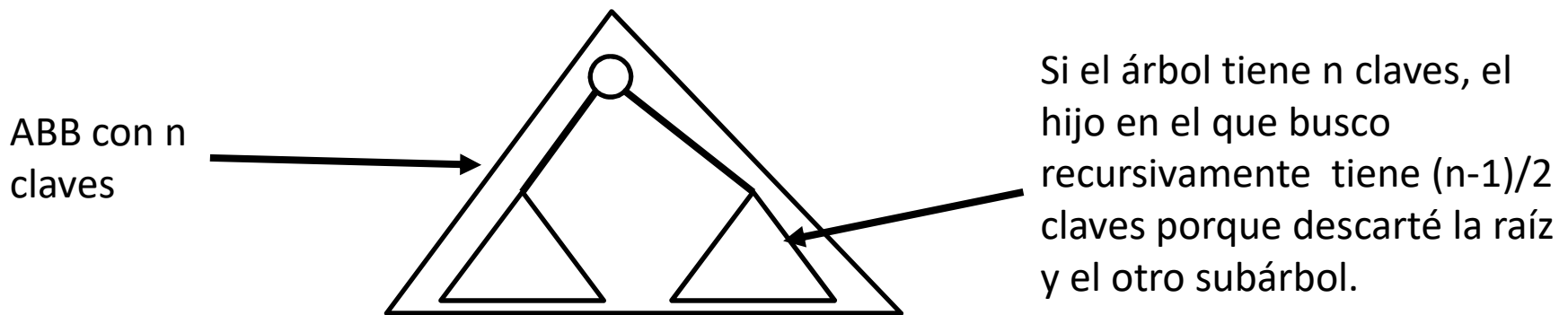
Entonces insertar otra clave en este árbol toma $O(h) = O(\log(n))$.

Análisis de la complejidad temporal

- Otra forma de estudiar el peor caso del mejor caso del ABB (que se da cuando el árbol está lleno y no encuentro la clave buscada):
- Sea n = cantidad de claves del ABB.

$$T(n) = \begin{cases} c_1 & \text{si } n = 0 \\ c_2 + T((n-1)/2) & \text{si } n > 0 \end{cases}$$

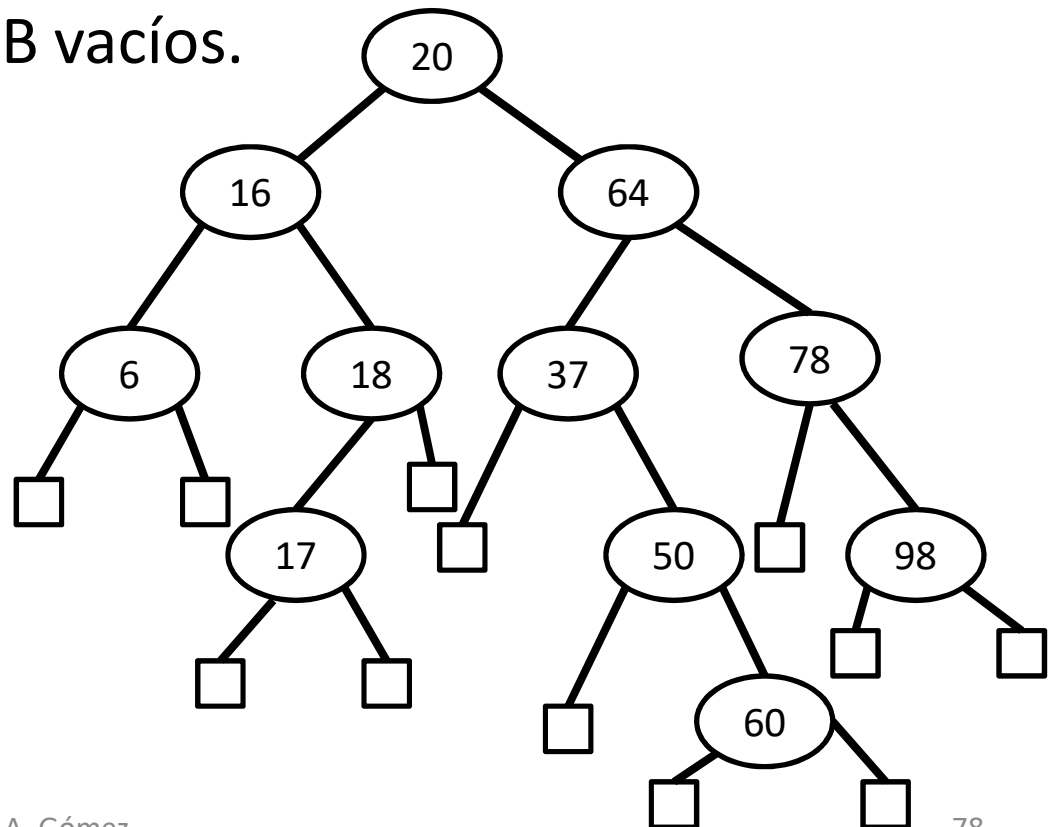
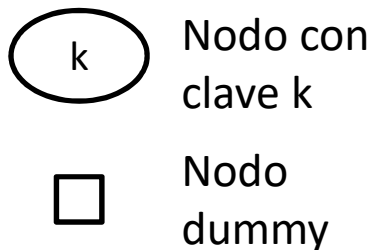
con lo cual $T(n) = O(\log_2(n))$.



Implementación en Java: Preliminares

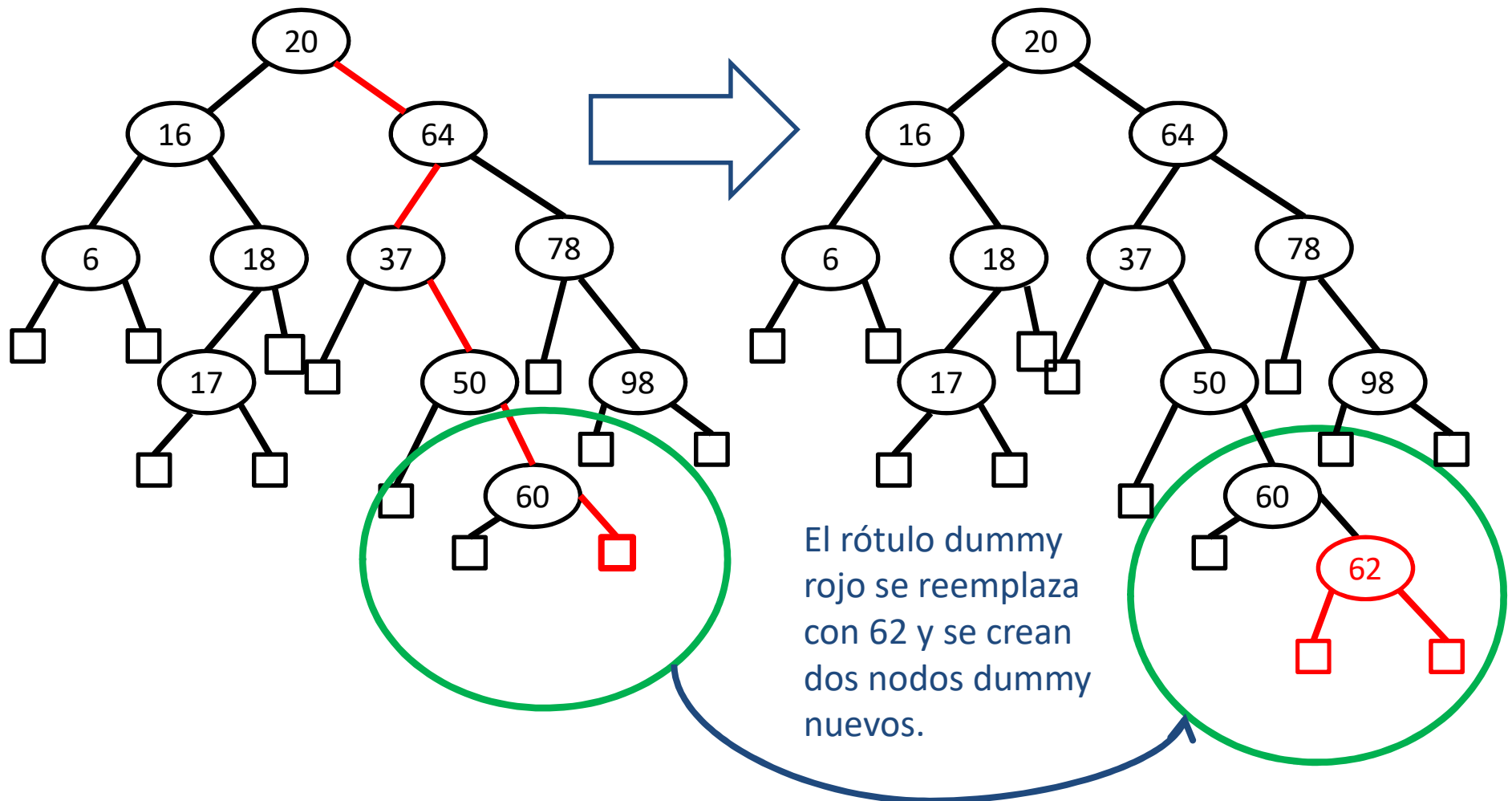
- Para facilitar la programación usaremos un árbol propio donde cada hoja tiene dos hijos dummy (placeholder según GT)
- Estos dummies sirven para insertar nuevas claves en su lugar y representan ABB vacíos.
- Ejemplo:

Referencias:



Implementación en Java: Preliminares

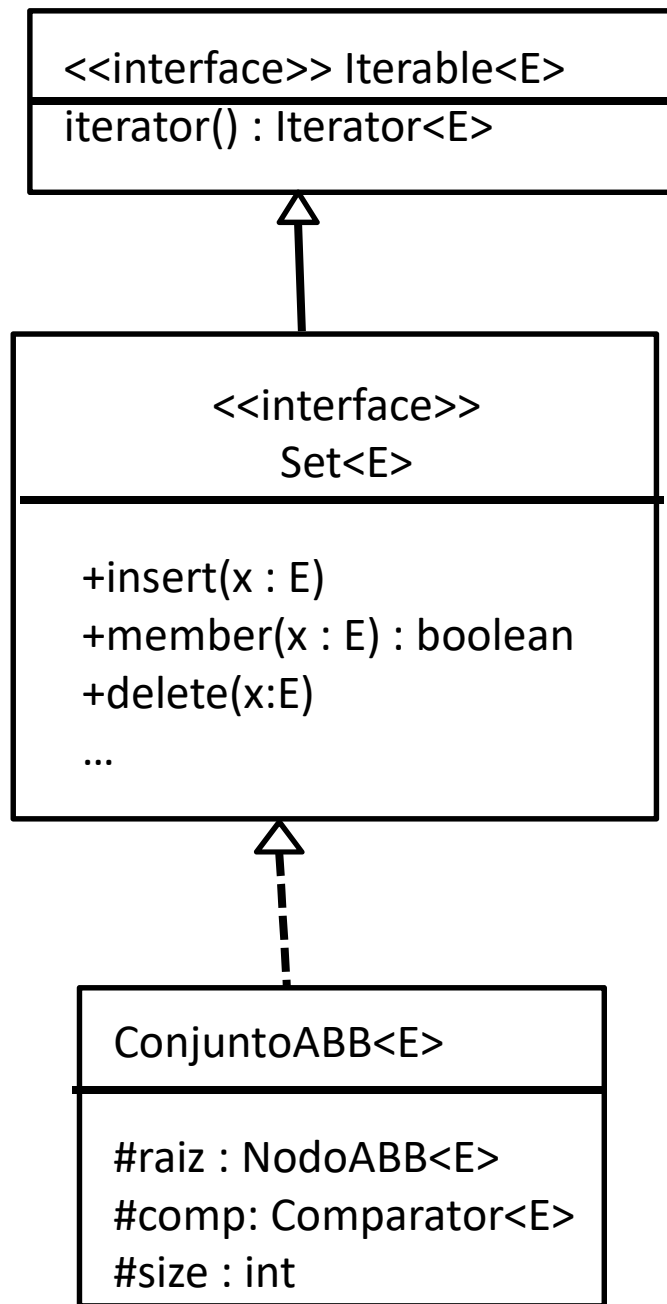
Supongamos que deseamos insertar el 62, para ello buscamos su ubicación:



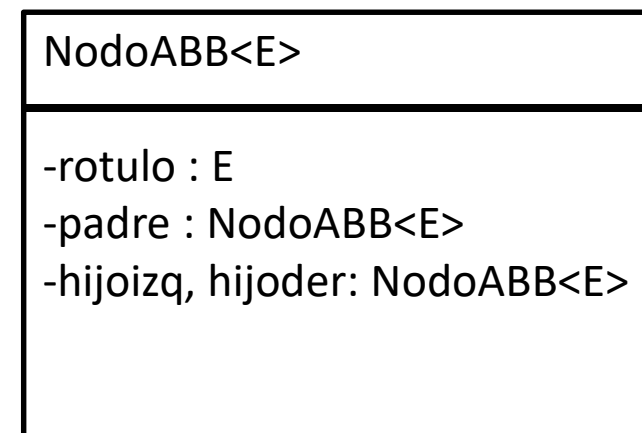
Implementación en Java: Preliminares

- ¿Cómo es el árbol vacío ahora?

Respuesta: El árbol vacío es un nodo dummy.



Primera aproximación a la implementación de un conjunto usando ABB:
Diagrama UML



Clase NodoABB

```
public class NodoABB<E extends Comparable<E>> {  
    private E rotulo;  
    private NodoABB<E> padre, izq, der;  
  
    public NodoABB( E rotulo, NodoABB<E> padre ) {  
        this.rotulo = rotulo;  
        this.padre = padre;  
        izq = der = null;  
    }  
  
    public E getRotulo() { return rotulo; }  
    public NodoABB<E> getPadre() { return padre; }  
    public NodoABB<E> getIzq() { return izq; }  
    public NodoABB<E> getDer() { return der; }  
  
    public void setRotulo( E rotulo ) { this.rotulo = rotulo; }  
    public void setIzq( NodoABB<E> izq ) { this.izq = izq; }  
    public void setDer( NodoABB<E> der ) { this.der = der; }  
    public void setPadre( NodoABB<E> padre ) { this.padre = padre; }  
}
```

Clase Conjunto ABB

```
import java.util.Comparator;

public class ConjuntoABB<E extends Comparable<E>> {
    protected NodoABB<E> raiz;
    protected int size;
    Comparator<E> comp;

    public ConjuntoABB(Comparator<E> comp) {
        raiz = new NodoABB<E>(null,null);
        size = 0;
        this.comp = comp;
    }
}
```

EJERCICIO: Programar este método usando iteración.

```
public boolean member( E x ) {
    return buscar(x).getRotulo() != null;
    // Si encontré a k, el rótulo del nodo no es null
}

private NodoABB<E> buscar( E x ) {
    return buscarAux( x, raiz ); // Búsqueda recursiva desde la raíz
}

private NodoABB<E> buscarAux( E x, NodoABB<E> p ) {
    if( p.getRotulo() == null ) return p; // Llegué a un dummy
    else { // Estoy en un nodo con un dato no nulo
        int c = comp.compare( x, p.getRotulo() );
        if( c == 0 ) return p; // Lo encontré
        else if( c < 0 ) return buscarAux( x, p.getIzq() ); // Busco a izq
        else return buscarAux( x, p.getDer() ); // Busco a derecha
    }
}
```

```

public void insert( E x ) {
    // Busco la ubicación de x en el árbol
    NodoABB<E> p = buscar( x );
    if( p.getRotulo() == null ) {
        p.setRotulo( x ); // Reemplazo null por el rotulo nuevo x
        p.setIzq( new NodoABB<E>( null, p) ); // Creo dummy a izq
        p.setDer( new NodoABB<E>( null, p ) ); // Creo dummy a der
        size++; // Tengo un elemento más y lo cuento
    }
}

```

Otra versión de insertar que no usa buscar

```
public void insert( E x ) {  
    insertarAux( x, raiz ); // Empiezo a buscar la ubicación de x desde la raíz  
}
```

```
private void insertarAux( E x, NodoABB<E> p ) {  
    if( p.getRotulo() == null ) { // Llegué a un dummy  
        p.setRotulo( x ); // Cambio el rótulo por x  
        size++; // Cuento un nuevo elemento  
        // Creo dos hijos dummy hijos izq y der con padre p:  
        p.setIzq( new NodoABB<E>( null, p ) );  
        p.setDer( new NodoABB<E>( null, p ) );  
    } else {  
        int c = comp.compare( x, p.getRotulo() );  
        if( c == 0 ) { /* x = rótulo de p => x ya estaba en el conjunto */ }  
        else if( c < 0 ) // x < rotúlo de p  
            insertarAux( x, p.getIzq() ); // Inserto en el hijo izq  
        else // x > rótulo de p  
            insertarAux( x, p.getDer() ); // Inserto en el hijo der  
    }  
}
```

EJERCICIO: Programar este método usando iteración.

Nota: El listado inorder de un ABB retorna las claves insertadas ordenadas en forma ascendente.

```
public String toString() {  
    return inorder( raiz );  
}  
  
private String inorder( NodoABB<E> p ) {  
    if( p.getRotulo() != null ) {  
        return "(" + inorder( p.getIzq())  
            + p.getRotulo()  
            + inorder( p.getDer() ) + ")";  
    } else return "";  
}
```

EJERCICIO: Programar el iterador del conjunto usando un recorrido inorder.

```

public class TestABB {
private static void insertar_imprimir( Set<Integer> s, int x ) {
    s.insertar( x ); System.out.println( "s.insertar("+ x + "): " + s ); }

```

```

public static void main( String [] args ) {
Set<Integer> t = new ConjuntoABB<Integer>( new DefaultComparator<Integer>() );
insertar_imprimir( t, 20 );
insertar_imprimir( t, 16 );
insertar_imprimir( t, 64 );
insertar_imprimir( t, 6 );
insertar_imprimir( t, 100 );
insertar_imprimir( t, 3 );
System.out.println( "Pertenece(18): " +
    t.pertenece(18) );
System.out.println( "Pertenece(100): " +
    t.pertenece(100) );
t.insertar2( 999 );
System.out.println( "t.insertar2("+ 999 + "): " + t );
t.insertar2( 4 );
System.out.println( "t.insertar2("+ 4 + "): " + t );
}
}

```

```

>java TestABB
t.insertar(20): (20)
t.insertar(16): ((16)20)
t.insertar(64): ((16)20(64))
t.insertar(6): (((6)16)20(64))
t.insertar(100): (((6)16)20(64(100)))
t.insertar(3): (((3)6)16)20(64(100)))
Pertenece(18): false
Pertenece(100): true
t.insertar2(999): (((((3)6)16)20(64(100(999)))))
t.insertar2(4): (((((3(4))6)16)20(64(100(999)))))

```

Eliminación: Dos estrategias

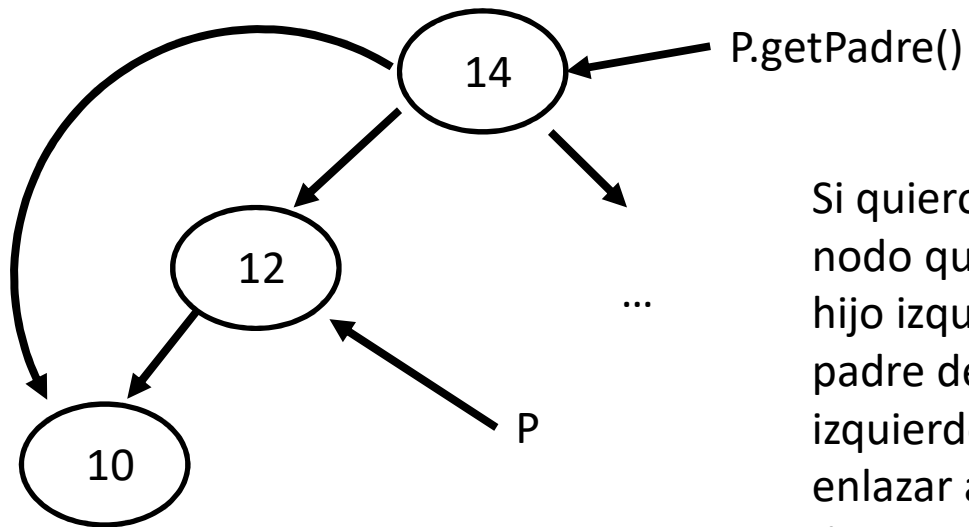
1. Eliminación perezosa (lazy delete): Busco el dato a eliminar y marco el nodo como eliminado (sin sacarlo del árbol). Para ello necesito un campo más en el NodoABB de tipo boolean que vale true si el dato ha sido eliminado. Member e Insert deben ser modificados para tener en cuenta esto; Cuando member encuentra el dato en el árbol debe además ver si el dato no está eliminado. Cuando insert encuentra el dato debe ver si no es necesario setear eliminado en falso.
2. Eliminación física: Busco el dato y elimino el nodo del árbol.

Eliminación física: delete(x)

- Paso 1: Buscar el nodo P con la clave x a eliminar
- Paso 2: Cuatro casos:
 1. P es una hoja: Eliminarlo
 2. P tiene sólo hijo izquierdo: hacer el bypass del padre de p con el hijo izquierdo de p
 3. P tiene sólo hijo derecho: hacer el bypass del padre de p con el hijo derecho de p
 4. P tiene dos hijos: Reemplazar el rótulo de p con el de su sucesor inorder y eliminar el sucesor inorder de p.

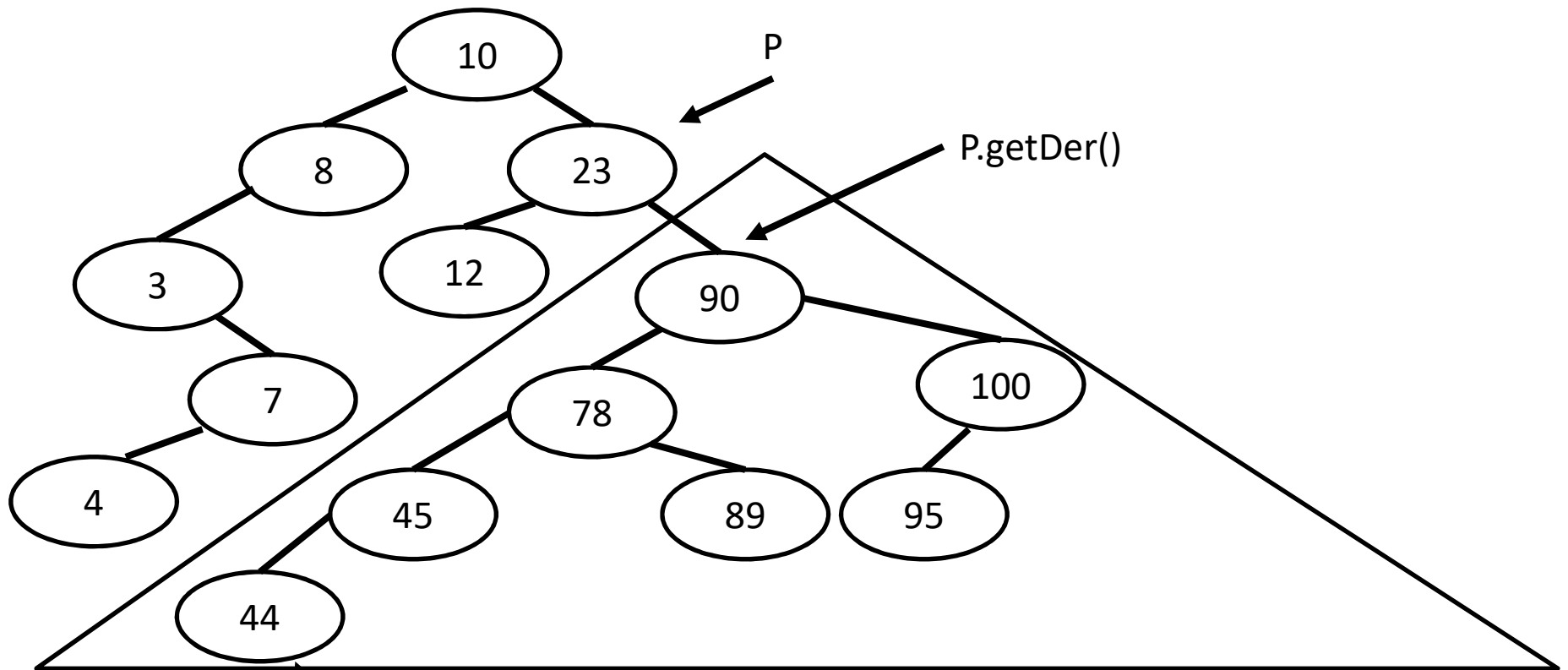
$$T_{\text{eliminar}}(h) = O(h)$$

Caso 2: ByPass



Si quiero eliminar al 2 y resulta que el nodo que contiene al 2 solo tiene hijo izquierdo, entonces debo subir al padre de 2, ver si 2 era el hijo izquierdo o derecho de su padre y enlazar al padre con el hijo izquierdo de 2.

Caso 4: Eliminar el mínimo valor del hijo de derecho de P



El mínimo valor de un subárbol se encuentra caminando hacia la izquierda todo lo que sea posible hasta llegar a un nodo sin hijo izquierdo.

El nodo encontrado es una hoja o solo tiene hijo derecho.
Eliminarlo cae en los casos (1) o (2).

```

// Retorna excepción null si no pudo eliminar a x
public void eliminar( E x ) throws SetException {
    // Busco x en el árbol para obtener el nodo p que lo contiene
    NodoABB<E> p = buscar( x );
    // Si el rótulo de p es null, p es un dummy, entonces
    // x no está en el árbol
    if( p.getRotulo() != null ) {
        // El rótulo de p no es nulo, entonces p contiene a x:
        // Invoco al método eliminarAux
        // que considera los 4 casos de la
        // eliminación de un nodo.
        eliminarAux( p );
        // Decremento size porque borré un nodo
        size--;
    } else throw new SetException("eliminar: No pude eliminar");
}

```

// p es hoja si sus dos hijos son dummies

```
private boolean isExternal( NodoABB<E> p ) {  
    return p.getIzq().getRotulo() == null &&  
           p.getDer().getRotulo() == null;  
}
```

// p sólo tiene hijo izquierdo si su hijo izq no es dummy y el derecho sí

```
private boolean soloTieneHijoIzquierdo( NodoABB<E> p ) {  
    return p.getIzq().getRotulo() != null &&  
           p.getDer().getRotulo() == null;  
}
```

```

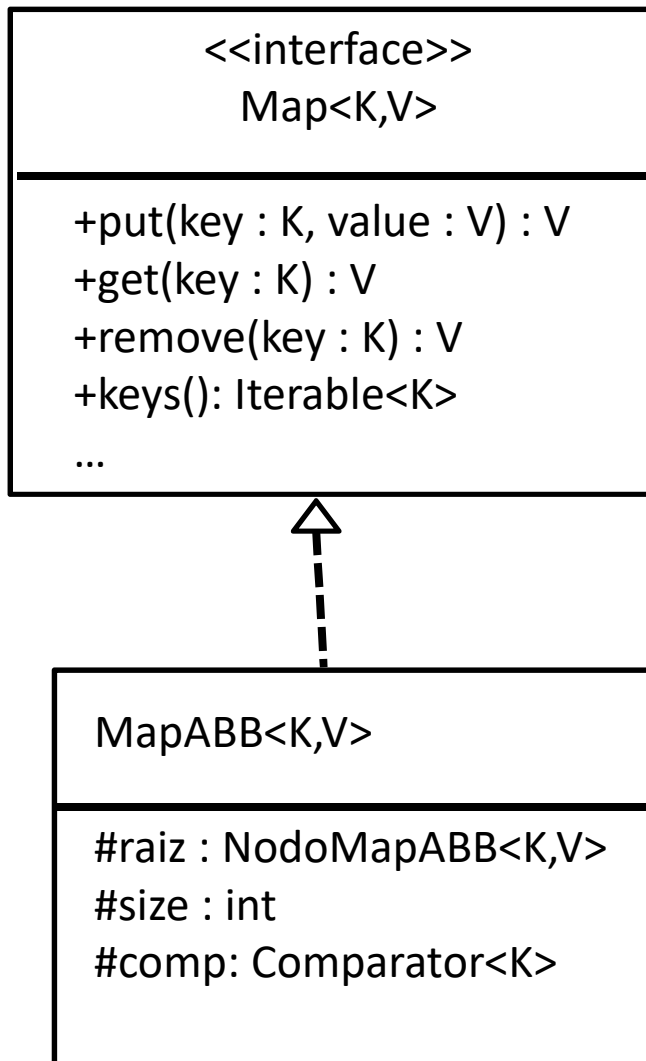
private void eliminarAux( NodoABB<E> p ) {
if( isExternal(p) ) {
    // Caso 1: p es hoja => Convertir el nodo en un dummy y soltar sus hijos dummy.
    p.setRotulo( null ); p.setIzq( null ); p.setDer( null );
} else { // p no es hoja
    if( p == raíz ) { .... Completar .... Con casos 2, 3 y 4 ... }
    else if( soloTieneHijolzquierdo(p) ) {
        // Caso 2: Enganchar al padre de p con el hijo izquierdo de p
        if( p.getPadre().getIzq() == p ) // p es el hijo izquierdo de su padre
            p.getPadre().setIzq( p.getIzq() ); // el hijo izq del padre de p es ahora el hijo de p
        else // p es el hijo derecho de su padre
            p.getPadre().setDer( p.getIzq() ); // el hijo derecho del padre de p es el hijo de p
            p.getIzq().setPadre( p.getPadre() ); // Ahora el padre del hijo izq de p es su abuelo
    } else if( soloTieneHijoDerecho(p) ) {
        // Caso 3: Enganchar al padre de p con el hijo derecho de p
        if( p.getPadre().getIzq() == p ) // p es hijo izquierdo de su padre
            p.getPadre().setIzq( p.getDer() ); // el hijo izq del padre de p es el hijo de p
        else
            p.getPadre().setDer( p.getDer() ); // el hijo derecho del padre de p es el hijo de p
            p.getDer().setPadre( p.getPadre() ); // Ahora el padre del hijo der. de p es su abuelo
    } else { // Caso 4: p tiene dos hijos => seteo como rótulo de p al rótulo del sucesor inorder de p.
        p.setRotulo( eliminarMinimo( p.getDer() ) );
    }
} // Fin else: p no es hoja
} // Fin método

```

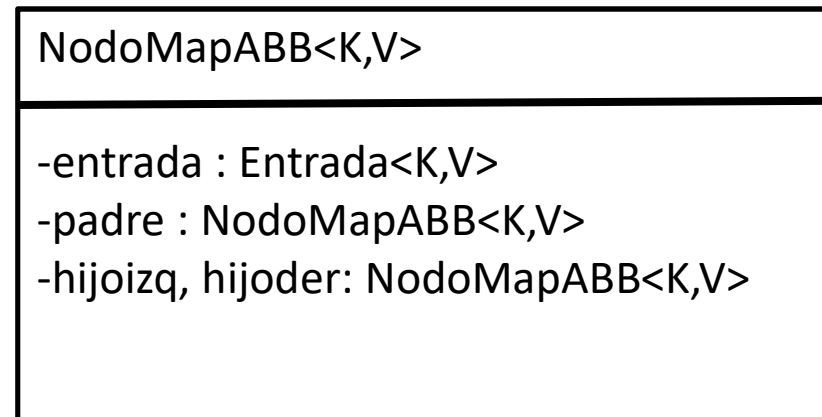
```

// Elimina el nodo con rótulo mínimo del subárbol que tiene como raíz a p
// El mínimo rótulo del subárbol que tiene como raíz a p es el rótulo del primer nodo que
// encuentro yendo a la izquierda que no tiene hijo izquierdo
private E eliminarMinimo( NodoABB<E> p ) {
    if( p.getIzq().getRotulo() == null ) { // El hijo izquierdo de p es un dummy
        E aRetornar = p.getRotulo(); // salvo el rótulo a devolver
        if( p.getDer().getRotulo() == null ) { // p es hoja (pues sus hijos son dummy)
            p.setRotulo( null ); // Convierto a p en dummy haciendo nulo su rótulo
            p.setIzq( null ); // y desenganchando sus dos hijos dummy
            p.setDer( null );
        } else { // p solo tiene hijo derecho (xq no tiene izquierdo)
            // Engancho al padre de p con el hijo derecho de p.
            // Seguro tiene que ser el hijo derecho de su padre.
            p.getPadre().setDer( p.getDer() );
            p.getDer().setPadre( p.getPadre() );
        }
        return aRetornar;
    } else { // Si p tiene hijo izquierdo, entonces p.getRotulo() no es el mínimo.
        // El mínimo tiene que estar en el subárbol izquierdo
        return eliminarMinimo( p.getIzq() );
    }
}

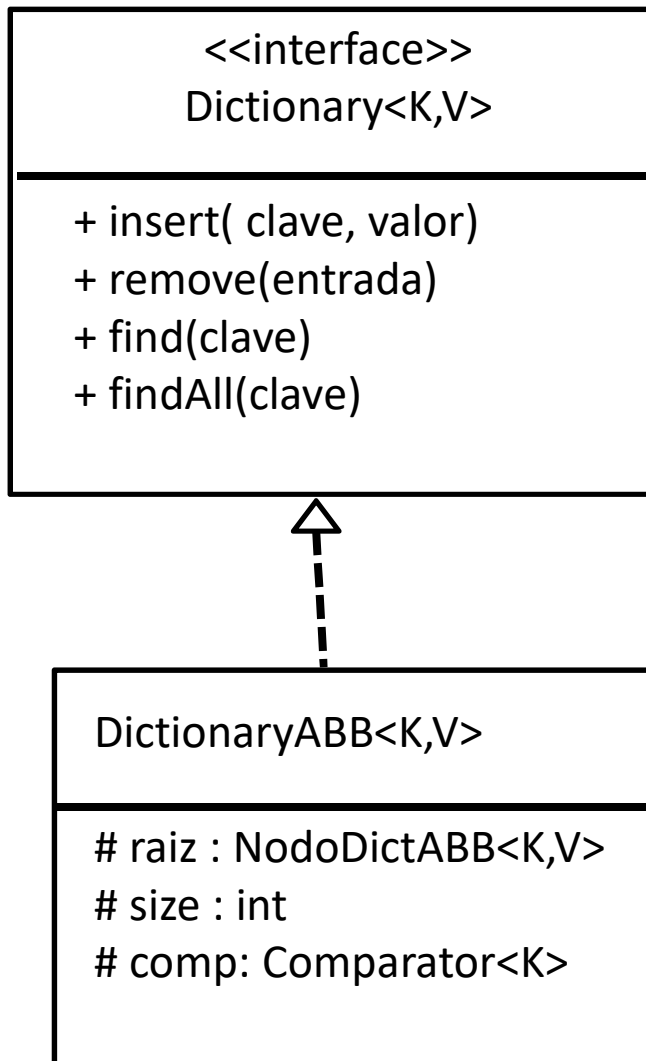
```



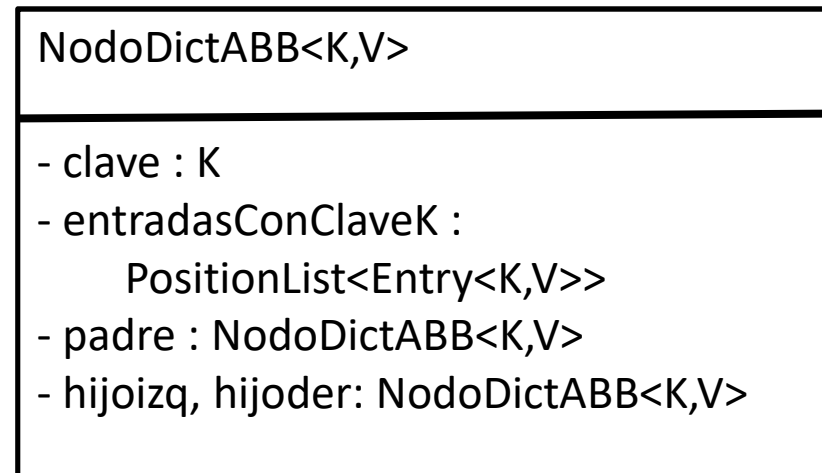
Discusión: Implementación de un mapeo usando ABB:



Dado un árbol de n entradas y altura h .
Get, put y remove tienen $O(h)$ ya que buscan ramificando por claves.
Keys, values, entries tienen $O(n)$ y usan listado inorder.



Discusión: Implementación de un diccionario usando ABB:



El nodo se crea cuando aparece la primera entrada con clave *clave*.
El nodo se destruye cuando se elimina la última entrada con clave *clave*.
La operación `find(clave)` retorna la primera entrada de la lista *entradasConClaveK* y `findAll(clave)` retorna la lista *entradasConClaveK*.

Discusión:

Implementación de diccionario con ABB:

Esquema parecido al mapeo pero ahora los nodos tienen una clave k y una lista de entradas con la esa clave k y el valor v_i correspondiente.

$$T_{\text{findAll}}(n) = O(h)$$

$$T_{\text{find}}(n) = O(h) \text{ donde:}$$

n = cantidad de entradas,

h = altura del ABB y

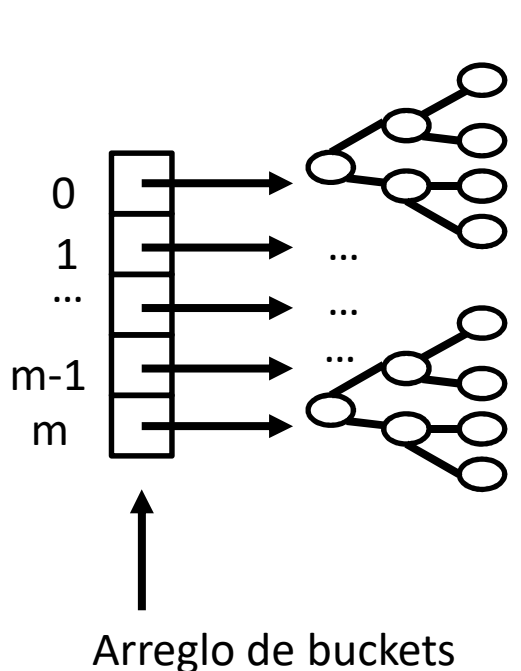
s = largo lista de entradas con una misma clave.

Si fuera necesario copiar la lista de entradasConClaveK entonces $T_{\text{findAll}}(n) = O(h + s)$

Optimizaciones a la tabla de hash abierto

Teníamos n entradas insertadas en un arreglo de m buckets donde cada bucket es una lista de entradas colisionadas. En lugar de estructurar los buckets como una lista, se pueden estructurar como ABBs.

Entonces:



¡El bucket i -ésimo ahora es un ABB y no una lista!

Nota: Ahora si por algún motivo perdí la distribución uniforme de claves tengo mejores chances de que los buckets grandes no se conviertan en listas muy largas.

Sin embargo, el peor escenario, aunque altamente improbable, es que todas las entradas terminen en un mismo bucket y encima vengán en orden ascendente, con lo cual el ABB va a ser una lista. Más adelante, veremos cómo resolver esto con "árboles de búsqueda balanceados".

Bibliografía

- Capítulo 10 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.